

Topographic Gating: Building Addressable Representations

Francesco Mannella

CNR, Institute of Cognitive Sciences and Technologies, Rome

September 24, 2026

Contents

1	Introduction	2
1.1	A representational space that can be used.	2
1.2	Topographic Gating.	3
1.2.1	From Self-Organizing Maps to Supervised Topological Maps.	3
1.2.2	Topographic Gating.	3
1.2.3	From a linear table to a nonlinear function.	5
1.2.4	Exact guarantee versus capacity-bounded inequality.	5
1.3	Continual learning as a consequence of Topographic Gating.	6
1.4	Generative replay and Topographic Gating.	7
1.5	How the rest of the paper is organized.	7
2	Architecture	7
2.1	Background: Generalizing SOMs to Topographic gating	7
2.2	Deep Generative Decoders and Topographic Gating	8
2.3	Maturity-based modulation	9
3	Continual Learning Experiments	10
3.1	Continual Learning via Topographic Gating	10
3.1.1	Protocol and classification readout	11
3.1.2	Anchor placement and generative replay in the training loop	11
3.1.3	Hyperparameters	11
3.1.4	Results	13
3.2	Interference in Topographic Gating	14
3.2.1	The learning-leakage bound	14
3.2.2	Experimental setting	15
3.2.3	Results	15
3.3	A Capacity Limit on the Interference Bound	16
3.3.1	The confusable-cluster diagnosis	16
3.3.2	Hypothesis: training dynamics	17
3.3.3	Hypothesis: routing	18
3.3.4	Hypothesis: capacity	19
3.3.5	Synthesis	21
4	Empirical Comparison to the Continual Learning Literature	21
4.1	Scenario and metric	21
4.2	Split-MNIST	22
4.3	Split-FashionMNIST	22
4.4	The closest peer group: SOM- and prototype-based methods	23
4.5	Beyond the 2019 and 2020 baseline literature	24

4.6	Comparability caveats	24
5	Discussion	25
5.1	Computational	25
5.1.1	Why topology, not just segregation.	25
5.1.2	New theoretical contribution: a two-term interference bound.	25
5.1.3	Topological alignment: this paper’s place in the larger picture.	26
5.2	Biological	26
5.2.1	Topographic organization is not a modeling convenience.	26
5.2.2	Separation for protection, continuity for generalization.	27
5.2.3	Scope of this claim.	28
5.3	Computational limitations	28
5.4	Biological limitations	29
5.5	Future Work	29
6	Conclusion	30
S	Supplementary Material	31
S.1	The STM kernel and energy	31
S.2	Kernel fusion	31
S.3	Single-phase learning-leakage bound	32
S.4	Two-term interference: learning leakage and generative-replay’s corrective term	33
S.5	Batch and online updates	34
S.6	Batch loss for the deep generative decoder	35
S.7	Which parameters Topographic Gating actually reaches	36
S.8	Architecture ablation: decoder width and channel count	37
S.9	Metrics	38
	References	38

1 Introduction

1.1 A representational space that can be used.

A learned internal representation is only as useful as what can be done with it once training ends. An implicit representation space, one with no record of where individual representations sit within it, cannot be queried, extended, or acted on by anything beyond the process that trained it. A reconstruction step run afterward, a rehearsal mechanism sampling from it, or any other process meant to be triggered by it, all need exactly that kind of access. Without an explicit layout of the space, representations stay locked inside the training loop that produced them.

We ask what is required for an internal space to have coordinates that are individually meaningful and addressable, and what advantages that property confers. Such spaces are called neural fields in the literature. In the computational vision literature, neural fields are networks whose weights parametrize a continuous field, taking a coordinate as input and returning the field’s value at that point, with no explicit storage of values (Xie et al., 2022). Coordinate-based networks are one such example, spanning photorealistic view synthesis (Mildenhall et al., 2020) and continuous 3D shape representation (Park et al., 2019); both applications need to query coordinates never observed during training. A more recent sub-literature trains such a space incrementally, as new views of the same scene arrive over a session (Po et al., 2023; Wang et al., 2024), applying rehearsal or distillation to the coordinate network once training has already happened. None of this addresses the harder requirement that a space keep discriminating among

a growing set of classes even though nothing tells it in advance, structurally, which region belongs to which class.

A second, older lineage reaches the same requirement through generative modeling rather than coordinate fields. The variational autoencoder (Kingma and Welling, 2013) and its adversarial refinement (Makhzani et al., 2015) regularize the encoder’s aggregate posterior toward a fixed prior, so that sampling from anywhere in that prior decodes to something plausible. The requirement is the same one stated above for coordinate fields, now imposed on a distribution as a whole, not on individual coordinates. The resulting space is dense and samplable by matching a density, with no guarantee that nearby points mean anything similar beyond what the prior’s own geometry happens to impose.

We add a further constraint, that coordinates encode spatial relationships meaningful by construction, and argue that this makes such representations easier for other processes to exploit.

1.2 Topographic Gating.

A space built to be queried at any coordinate is, by that same construction, a space with a topology: positions have neighbors, and neighbors carry a well-defined notion of distance. A neural field can be built with that topology left implicit (see Figure 1a): in a NeRF, for instance, which coordinates end up behaving similarly to which others is whatever the trained weights happen to encode, with no structure imposed in advance on the relationship between coordinates and representations. We build on that idea here by constraining the topology of the representation space to a neural topography, making the relationship explicit. A neural topography is a fixed geometric relationship between the units of an inner neural layer. These units are laid out, for instance, on a grid. In this layout, distance between units is defined in advance. It is not left for training to discover. Position in that fixed layout is then used directly to control training itself, deciding, unit by unit, how much of any given training signal is allowed to reach it.

1.2.1 From Self-Organizing Maps to Supervised Topological Maps.

The starting point is a simple structural idea. Arrange the units of some inner layer so that they carry a fixed neighborhood structure, a topography. Then let each unit’s position in that structure gate how much of the training signal for a given example reaches it. Units near the position responsible for the current example receive most of the update, while units far from it receive almost none. Repeated across many examples, this is by itself sufficient to drive nearby units toward similar responses, without specifying the categories beforehand. That principle is long-standing and well-studied, known as self-organizing maps (SOMs; Kohonen, 1982, 2001). Mannella (2020) added a further step: instead of only gating updates around wherever the current example’s evidence happens to land, also gate them around a target position fixed ahead of training for the example’s class. That anchor-driven step, *Topographic Gating*, defines the Supervised Topological Map (STM). It implies active control over which units of an inner layer’s topography a given training example can update.

Neither classical k-means nor the self-organizing map, in conventional use, is a proper neural field. In k-means, the training update is hard. In the SOM, it is diffused over a neighborhood. But both still settle on a hard readout: a query returns one unit’s own stored row via best-matching-unit lookup, never a blend across neighbors. A value exists only at a unit actually retrieved this way. Querying anywhere in between returns nothing: no function is defined there, only a finite set of stored points. Both methods implement a discrete vector-quantization scheme, not a continuous field.

1.2.2 Topographic Gating.

In STM, training is controlled by pointing the gate at a position fixed before training begins. The mechanism composes two fields: the neighborhood of the BMU relative to an input pattern,

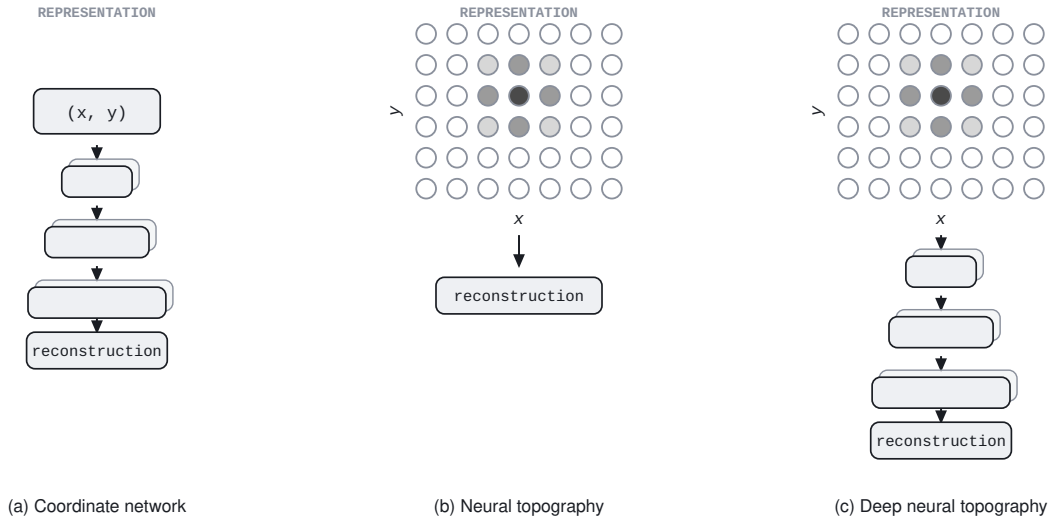


Figure 1: Three ways to build a representational space that can be queried at any coordinate, differing in whether the space’s topology is implicit or explicit and whether the readout from it is shallow or deep. (a) *Coordinate network* (e.g. a NeRF): a raw coordinate is passed through a deep nonlinear stack whose intermediate layers carry no imposed topology; whatever structure exists among them is a byproduct of training, not a design choice. (b) *Neural topography*, shallow: the representation layer is instead a grid of units with a fixed neighborhood structure, a neural topography, so a given input activates a local population of units, population coding, rather than a single coordinate value; readout is the single linear layer directly addressed by grid position. (c) *Deep neural topography*: the same explicit grid topography as (b), but the grid vector is passed through a deep nonlinear decoder rather than read out linearly, combining explicit topology with the expressive capacity of (a).

and the radial distance from an anchor associated with the subset that pattern belongs to. Figure 2 shows the gate as the product of these two fields over an explicit grid. Topographic Gating combines controlled gating during learning with a smooth-query mechanism for readout. Queries to the readout network are built as a Gaussian-weighted blend over the grid’s units. The resulting interpolation ensures the readout is a continuous function from map coordinate to reconstructed feature vector, defining a neural field (Section 1.1). The mechanism generalizes the approach used by hybrid neural fields such as feature-grid architectures (Müller et al., 2022). For shallow networks like STMs, it reduces to a discrete grid of stored vectors made continuous through interpolation (Figure 1b). The same choice also draws on an older body of literature beyond vision and graphics, where what is termed a neural topography here closely aligns with what neuroscience refers to as a neural field. In this literature, a neural field is a population of units arranged over an explicit, architecturally fixed space, where a lateral-interaction kernel scales how one position’s activity depends on distance to another (Amari, 1977; Wilson and Cowan, 1972). That definition of neural field was first used to model sensory encoding and later extended to abstract, non-spatial feature spaces relevant to cognition and motor control in Dynamic Field Theory (Erlhagen and Schöner, 2002). The neural topography introduced in this paper serves as a conceptual bridge between two traditions that seldom intersect. On one side stand coordinate networks, in the visual-computing sense, whose topology remains implicit, emerging solely from whatever structure the trained weights happen to encode. On the other stand lateral-inhibition-kernel networks, as understood in neuroscience, whose topology is constrained a priori by a preexisting neural topography.

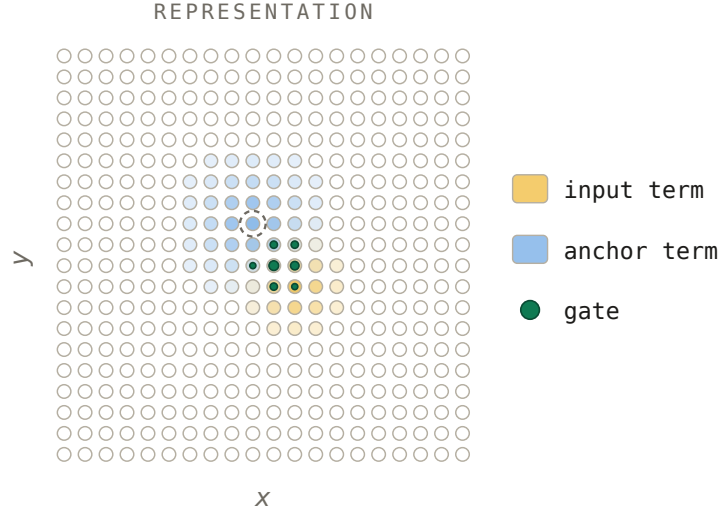


Figure 2: Topographic Gating on an explicit grid. The gate, which sets how much grid unit j is updated by training example i , is the product of two Gaussian fields over the grid: an *input term* centered on the unit that currently responds best to the example’s evidence (gold), and an *anchor term* centered on the position fixed ahead of training for the example’s class (blue, dashed ring). Their product, the gate itself (green), is appreciable only where the two fields overlap, so an example is allowed to change the grid only where current evidence and the committed anchor agree. The gated grid vector is then read out through the deep nonlinear decoder of Figure 1c to a reconstruction.

1.2.3 From a linear table to a nonlinear function.

We extend this approach further here, replacing independent rows with a single nonlinear function of grid coordinates. Here, the Gaussian filter over coordinate space is no longer used to construct a direct weighted average of prototypes in a lookup table. Instead, it functions as a soft query to a deep network (Figure 1c). The deep network functions as a generative decoder with Topographic Gating. Topographic Gating here determines which part of the low-dimensional space, defined by the shared function, each class updates. The gate aligns the function’s parameters with the class being trained by combining competition for the best-matching position, cooperation among neighboring positions, and a fixed target position per class. Training is governed by a single continuous function mapping topographic coordinates to prototypes, enabling any update to propagate influence across the entire function. Protection is now based on the geometry of update location: only the decoder’s first layer is influenced by topographic position and here training is gated by the chosen anchor; the hidden weights and filters above it are shared across classes and receive gradients in every phase, at every topographic distance. Soft queryability and accuracy can be separated as different effects of this mechanism. Queryability comes from interpolation alone, as already established: the same Gaussian-weighted blend that makes the readout a continuous function of grid coordinate, defined and smoothly varying between committed grid positions, not only at them. Accuracy depends on the capacity of the readout function to shape each reconstruction precisely.

1.2.4 Exact guarantee versus capacity-bounded inequality.

During training, topographic gating does not enforce hard control. Exclusion from gating is not binary but graded: a unit far from the class currently being trained still receives a small, nonzero update, scaled by how far it sits from that class’s anchor. The paper calls this residual, unwanted update *learning leakage* and bounds how large it can grow as a function of that distance (the

learning-leakage bound).

The deep decoder’s enhanced capacity comes at a structural cost. Only its first layer maintains grid-position specificity; every subsequent layer, along with every convolutional filter in the upsampling stack, is shared across all classes and continues training throughout every phase, regardless of topographic distance. Parameters shared this broadly accumulate a small, unconstrained influence from every class the network encounters, and this is the source of learning leakage. A class assigned too little of it drifts, and that drift progressively encroaches on neighboring classes. Later sections show that separation is set by model capacity, the representational resource below the topography, and not by the training schedule. With enough capacity, the decoder keeps classes far enough apart that this drift rarely crosses into confusion, so leakage rarely produces an actual classification error. Dedicated experiments later in the paper show that sufficient capacity outperforms eliminating this leakage entirely.

1.3 Continual learning as a consequence of Topographic Gating.

Protection against catastrophic forgetting is one consequence of an addressable, gated space.

A continual learning system, by definition, must sequentially acquire new classes or tasks while preserving previously consolidated knowledge, all within a fixed parameter budget and without unrestricted access to historical data. Any mechanism addressing this constraint must determine, implicitly or explicitly, which parameters are permitted to change for a new task and which must remain protected. The literature typically resolves this by safeguarding *parameters* rather than representation, with protection retrofitted after the fact onto architectures that possess no native notion of "this parameter belongs to this task." EWC (Kirkpatrick et al., 2017) and SI (Zenke et al., 2017) estimate, post-hoc, which parameters were critical for prior tasks and penalize their modification; GEM (Lopez-Paz and Ranzato, 2017) and A-GEM (Chaudhry et al., 2019) instead constrain each gradient update against a buffer of prior examples; PackNet (Mallya and Lazebnik, 2018) and HAT (Serra et al., 2018) go further still, partitioning the network itself into disjoint, frozen per-task subsets. Each is a bookkeeping mechanism compensating for the same underlying limitation: the parameters themselves encode no intrinsic representation of task identity.

Topographic Gating removes the need for this bookkeeping entirely. Protection is intrinsic to the topography that makes the space addressable, keeping the internal representation intact. The mechanism underlying this protection is maturity-based modulation, whereby each class is assigned to an explicit, addressable position within the grid—an anchor—rather than to an arbitrary, non-localized set of parameters. The anchor formalizes the current learning state of a class as a well-defined, trackable quantity. A maturity trace quantifies the degree of alignment between a class’s samples and its designated anchor. Once a class reaches high maturity, its neighborhood bandwidth is pulled ahead of the shared annealing schedule, toward the value that schedule will eventually reach. Consequently, a well-learned class is protected earlier than a poorly-learned one. The present formulation adapts a per-node maturity mechanism from a broader cross-modal alignment framework (Zubek et al., 2025), which rests on the same basic premise: only an explicit, addressable representation makes a per-unit maturity signal meaningful in the first place. That framework calls this same per-node quantity *competence*, since it gates a policy’s exploration noise and reflects that policy’s demonstrated ability to produce effects; this paper calls it *maturity* instead, because the classification-only setting studied here has no policy or action at all, and the quantity tracks only how settled a class’s own representation currently is. The formal specification of the maturity-modulation mechanism appears later and is used in every continual-learning experiment reported in this paper.

Anchor geometry bounds how much training on one class can influence representations associated with another. Target positions are placed with sufficient mutual separation across the topography, and by the time a class has converged in this maturity-based sense, the gate around its target has narrowed to a radius that does not reach another class’s target position.

Training on a newly introduced class then reaches previously learned classes’ representations only through the gate’s own small, bounded pull. This argument is formalized later, accompanied by the corresponding empirical validation.

1.4 Generative replay and Topographic Gating.

Generative replay is a feature of the architecture. The same decoder that produces the bound described in Section 1.2.4 is also generatively queryable at any anchor-relative grid coordinate, the operation Topographic Gating already requires for the inverse mapping from grid position to reconstructed sensory pattern. Because this queryability is already built into the architecture, generative replay removes the need for a second network; rehearsal samples are decoded directly from the model currently being trained. Here is where it differs from deep generative replay (DGR) (Shin et al., 2017). In DGR the generator is a separate network, trained alongside the task solver purely to support rehearsal and itself subject to forgetting as it is retrained on each new task. No auxiliary generator is employed here; rehearsal samples emerge directly from the substrate, which remains continuous and generatively queryable. We exploit this capability to improve continual-learning accuracy. For previously seen classes we inject decoder-sampled synthetic items into each phase’s training set, drawn from the model’s own generative capacity rather than from stored real examples. Parameters that remain exposed to cross-class learning leakage regardless of grid distance accumulate a small drift over a run; rehearsing earlier classes pulls those parameters back toward their earlier values, recovering in practice some of what the shared decoder’s inequality permits in principle. Generative replay’s actual contribution, however, is small. A later section runs a direct continual learning test: it disables generative replay, keeps every other setting fixed, and applies Topographic Gating alone on the generative decoder. Accuracy stays close to the full configuration that includes generative replay during training. The residual drift replay corrects is quantified formally later; topography alone already accounts for most of the anti-forgetting effect reported in this paper.

1.5 How the rest of the paper is organized.

Section 2.2 gives the deep, shared decoder its full architectural treatment, replacing the shallow lookup table described above with a nonlinear function of grid position, and specifies the single-layer model of Mannella (2020) and the shallow baseline used for comparison throughout. Section 3.1 lays out the continual-learning protocol, including how decoder-sampled synthetic images are injected into each phase’s training set for replay. Section 3.2 makes the geometric-protection argument precise, turning the shared decoder’s cross-class sharing into a formal bound on prototype drift, with an empirical test (Section 3.2.3) isolating generative replay’s own, modest contribution from Topographic Gating’s. Section 3.3 takes up the question the interference bound leaves open, what determines whether a given drift becomes an actual error, and finds the answer in model capacity; the corresponding isolating-control experiments (Section 3.3.4) confirm that reverting to an unshared table costs more than it protects.

2 Architecture

2.1 Background: Generalizing SOMs to Topographic gating

Self-organizing maps (SOMs; Kohonen, 1982, 2001) are a classic unsupervised algorithm for arranging a grid of prototype vectors so that nearby grid positions come to represent similar inputs. Like k-means (MacQueen, 1967), each input is assigned to its closest prototype, the winner; unlike k-means, that prototype’s spatial neighbors on the grid are pulled toward the input as well. Repeated over many inputs, this competitive-cooperative update self-organizes the grid into a topology-preserving map without ever being told in advance what categories exist.

The supervised topological map (STM) of Mannella (2020) keeps this same mechanism but adds supervision. Each input carries, in addition to its own evidence, a target grid position, an anchor, assigned in advance according to its class, and the update is pulled toward both the winner’s neighborhood and the anchor’s neighborhood at once. The class an input belongs to determines, before training even sees that input, which region of the grid its update is allowed to reach: a topology that the plain SOM only discovers implicitly during training is here imposed directly. k-means, the SOM, and the STM in fact share a common structure: all three are the same energy-minimizing operator under three different weightings, defined next.

The shared representation space is a $K \times K$ grid of nodes, $Z = \{1, \dots, K^2\}$, each node j carrying a fixed grid position $p_j \in \mathbb{R}^2$; distances between nodes are Euclidean in this grid space. Given a set of inputs $\{x_i\}$ and a prototype $w_j \in \mathbb{R}^D$ for every node, k-means, self-organizing maps (SOM), and the supervised topological map (STM) of Mannella (2020) all minimize the same weighted-distortion energy

$$L(W) = \frac{1}{2} \sum_i \sum_j \varphi_{i,j} \|x_i - w_j\|^2,$$

and differ only in how the weight $\varphi_{i,j}$ is defined. Writing the *best-matching unit* (BMU) of input x_i as $r_i = \arg \min_k \|x_i - w_k\|$, the three members of the family are:

Algorithm	$\varphi_{i,j}$	Character
k-means	$\mathbb{I}[j = r_i]$	hard, no topology
SOM	$\exp(-\ j - r_i\ ^2 / 2\sigma_r^2)$	soft, topology, unsupervised
STM	$\exp(-\ j - r_i\ ^2 / 2\sigma_r^2) \cdot \exp(-\ j - t_i\ ^2 / 2\sigma_t^2)$	soft, topology, supervised

k-means assigns all update mass to a single winner. SOM diffuses it over a neighborhood of the winner, producing topological continuity (nearby nodes respond to similar inputs) with no supervision at all. STM adds a second factor that additionally requires the update to respect a pre-assigned target position t_i in the shared representational space for input x_i ’s class.

Regardless of the choice of $\varphi_{i,j}$, all three members of the family share a common architecture: each is implemented as a single linear layer—a (K^2, D) weight matrix W indexed directly by grid code, equivalent to a lookup table (Figure 3).

The STM operator, denoted $\varphi_{i,j}^{\text{STM}}$ throughout the paper, is SOM’s neighborhood term multiplied by an anchor term. The anchor term imposes a further constraint, suppressing the update for any node that lies far from the target t_i assigned to the input’s class, however close that node sits to the winner. This is stated formally as a definition later, together with the corresponding energy’s convexity properties and the two limiting cases that recover SOM and a label-modulated k-means.

2.2 Deep Generative Decoders and Topographic Gating

In the shallow, single-layer case (Section 2.1), each unit j owns an independent, directly-stored prototype w_j . The deep generative decoder replaces this table with a single trainable function. A network $\text{dec}(\cdot)$ takes a K^2 -dimensional grid vector as input and produces a reconstruction. Each unit’s prototype is computed on demand: $w_j = \text{dec}(e_j)$, the network’s output for the one-hot vector e_j at position j . During training, the network is instead queried at z_i : the row-normalized vector of $\varphi_{i,j}^{\text{STM}}$ values for sample i (i.e. $z_i[j] \propto \varphi_{i,j}^{\text{STM}}$). This vector approximates a one-hot indicator at i ’s best-matching unit.

The decoder used in this paper’s experiments follows a short pipeline (Figure 4). A single linear layer projects the K^2 -dimensional grid vector to a small feature map. A stack of transposed-convolution layers then upsamples this feature map to the final image resolution. Routing the grid vector through a hidden bottleneck before the convolutional stage is decisively worse than

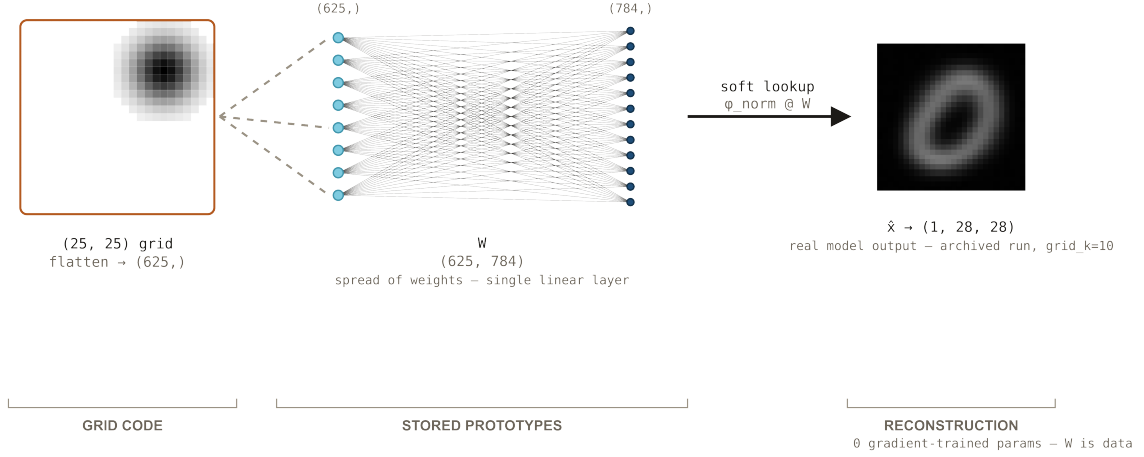


Figure 3: The generic architecture shared by the winner-take-all family: a grid code is looked up directly against a stored (K^2, D) prototype matrix W (a single linear layer with no hidden units and no nonlinearity), yielding a weighted average $\tilde{\varphi} \cdot W$.

direct projection. The projection therefore remains a single linear layer throughout. Only the convolutional stage’s channel count remains an open architectural choice; later results are reported across several settings. The pipeline’s overall shape is otherwise fixed: grid vector in, image out, through a network whose parameters are shared across every unit rather than owned by any one of them.

Sharing parameters across units gives the decoder a generative, interpolable readout: any grid coordinate can be queried, beyond the K^2 committed prototypes. The shallow, single-layer network never had to address how far Topographic Gating’s protection extends into a network’s internal weights, as opposed to stopping at the prototypes it outputs. Layer by layer, the decoder’s first layer is gated exactly as the shallow baseline’s prototype rows are (Section 1.2.3); every layer downstream of it, including each convolutional layer in the upsampling stack, does not inherit that property, because gradients through a shared function mix contributions across units.

The geometric-protection property itself, that a class’s target position gates how much a given update can affect another class’s prototype, has a precise quantitative form. The STM kernel that gates every update fuses into a single Gaussian of fixed spatial width, giving a bound on prototype drift that decays doubly with grid distance to the anchor. This fused-kernel result is then used to derive and test the interference bound this paper’s anti-forgetting claim rests on.

2.3 Maturity-based modulation

Topographic Gating gives every class an explicit, addressable position in the grid: an anchor t_i that pins the class to a specific region of the representational space (Section 1.3). This anchor makes a further mechanism possible: online, per-class modulation of the neighborhood bandwidth σ_r , adapted from the per-node maturity modulation introduced for the broader cross-modal alignment framework (Zubek et al., 2025). Tracking how well a class’s own samples currently

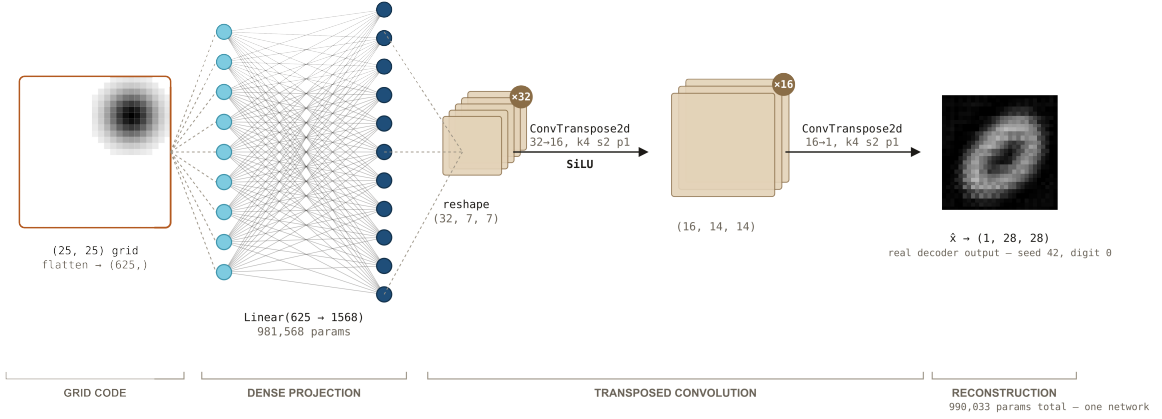


Figure 4: The deep generative decoder used throughout this paper’s experiments ($K^2 = 625$, `decoder_hidden=0`, `decoder_channels=24`). A 625-dimensional grid vector is projected by a single dense layer to a (24, 7, 7) feature volume, then upsampled by two transposed-convolution layers to a 28×28 reconstruction. The same forward pass, using a single decoder network, produces every prototype ($W = \text{dec}(I_{625})$), every reconstruction, and every generative-replay sample. Compare against Figure 3: this entire pipeline replaces the shallow baseline’s single stored matrix.

match its anchor requires that the class already have an explicit location to measure against. An architecture without an explicit topographic representation of classes has no natural per-class site for such a trace, nor a local neighborhood whose bandwidth could be selectively narrowed. A per-class maturity trace c_{mat} is updated online as an exponential moving average of how well the class’s own samples currently match their anchor, and a global maturity C_{tot} , the grid-wide average of that trace, is derived from it:

$$\sigma_r^{(c)} = \sigma_r^{\text{final}} + (\sigma_r - \sigma_r^{\text{final}}) \cdot (1 - \tanh(\beta C_{\text{tot}}))(1 - \tanh(\beta_{\text{local}} c_{\text{mat}})),$$

with fixed decay constants β and β_{local} . High maturity (both terms near 1) collapses $\sigma_r^{(c)}$ toward the already-annealed floor σ_r^{final} for that class, so later phases (each training phase is defined formally later) perturb a well-learned class’s neighborhood less than a poorly-learned one’s. Zubek et al. (2025)’s full mechanism additionally modulates a learning rate and a policy-exploration noise term. A classification-only setting has no policy to explore, so the implementation used here applies only the σ_r modulation, used throughout every continual-learning experiment in this paper.

3 Continual Learning Experiments

3.1 Continual Learning via Topographic Gating

We test whether Topographic Gating, combined with maturity-based modulation (Section 2.3) and the anchor-placement and generative-replay mechanisms described below, is sufficient for continual learning under a standard class-incremental benchmark. The measure is how much accuracy survives training on a sequence of classes, and how much is lost to forgetting.

3.1.1 Protocol and classification readout

All continual-learning experiments in this section use the Class-Incremental Split-MNIST protocol (van de Ven and Tolias, 2019), the primary empirical test of the geometric anti-forgetting argument introduced in the Introduction (Section 1.3); the same protocol is later applied to Split-FashionMNIST and, preliminarily, Split-CIFAR-10. The ten MNIST digit classes are split into five binary phases, $\{0, 1\}$, $\{2, 3\}$, $\{4, 5\}$, $\{6, 7\}$, $\{8, 9\}$, presented sequentially. Each phase trains exclusively on its own digit pair for a fixed number of epochs (20). The five-phase sequence is traversed once; no phase is revisited outside of the model’s own anchor and replay mechanisms. Held-out evaluation uses the real MNIST test split (train/validation, used only for fitting, come from a 95/5 split of the MNIST training set).

Because the model itself has no classifier head and produces only prototypes and reconstructions (Sections 2.1 to 2.2), evaluation needs an explicit rule for turning a test image into a predicted digit label. The model classifies by *majority-vote best-matching-unit labeling*, finding the test image’s best-matching unit (Section 2.1) against the current prototype set and reading off that unit’s assigned label. A grid unit’s assigned label is recomputed after every phase, as the digit class whose training samples most often had that unit as their own best-matching unit, counted only over classes trained so far. Evaluation throughout this section is class-incremental. No task identity is available at test time, and this per-unit labeling is always built over the full running set of classes trained so far, never a task-restricted subset. Class-incremental is the harder and more realistic of the standard continual-learning evaluation scenarios (van de Ven and Tolias, 2019), compared to Task-IL or Domain-IL, which retain task identity or a per-task output head at test time. Results below are therefore reported in the matching Class-IL metric, and this paper’s later literature comparison also uses Class-IL as its basis.

Results below are reported in three metrics standard in the continual-learning literature (Lopez-Paz and Ranzato, 2017). AA is the average per-task accuracy after the final phase. BWT is the average change in a task’s own accuracy between when it was learned and the end of training (backward transfer, i.e. forgetting). Class-IL is a single pooled accuracy over the full ten-class test set, as opposed to AA’s unweighted per-task average. The exact formulas appear later.

3.1.2 Anchor placement and generative replay in the training loop

The mechanisms of Sections 1 and 2.2, together with the interference-protection argument developed later, tie together in the phase structure above as follows. All K^2 anchor positions are initialized once, before phase 0, by a greedy, distance-weighted seeding procedure that spreads the ten initial anchors apart across a uniform sample of grid coordinates. Each class’s anchor is then re-placed online, at the start of *its own* phase. This reuses the same per-unit tally used for classification for a different purpose, finding which grid units currently receive the most best-matching-unit hits from the class about to be trained, placing that class’s anchor toward the center of that region, and pushing it away from already-committed anchors by a repulsion term. Generative replay fires every phase after the first. It samples one replay batch from the decoder at every previously-seen class’s own (jittered) anchor before each phase’s training loop starts, and combines it, batch by batch, with the current phase’s real data, directly implementing the $p_c/p_{c'}$ split detailed later.

3.1.3 Hyperparameters

Table 1 gives the full hyperparameter configuration. One aspect is not simply read off the table: anchors are not fixed in advance but re-placed online, once, at the start of each class’s own phase, by majority-vote probing of which grid units currently respond to that class, combined with a repulsion term that keeps the new anchor apart from anchors already committed to earlier classes, as described above.

Table 1: Hyperparameters for the Class-Incremental Split-MNIST experiments of Section 3.1.

Setting	Value
Grid and decoder (Section S.8)	
Grid size K^2	625 ($K = 25$)
Decoder architecture	convolutional
Decoder hidden layer	none
Decoder channels	24
Neighborhood and anchor bandwidth	
σ_r schedule shape	exponential, front-loaded (exponent 0.3)
σ_r : initial / mid / final	$2.0 \rightarrow 1.0 \rightarrow 0.5$
σ_t (anchor bandwidth)	0.8, fixed (no annealing)
Anchor placement (Section 3.1.2)	
Timing	online, once per class’s own phase
Method	majority-vote probing and repulsion from committed anchors
Repulsion weight	2.0
Repulsion radius	3.375 grid units
Border repulsion	$0.3 \times$ repulsion weight
Classification and replay	
Classification readout	majority-vote BMU labeling (Section 3.1.1)
Generative replay	on
Replay volume	$1.5 \times$ current phase’s real data
Anchor-position jitter	0.5 grid units
Maturity modulation (Section 2.3)	
Status	on
Decay constant β (global)	3.0
Decay constant β_{local}	1.0
Loss and optimization	
STM-energy weight	1.0
Pixel-reconstruction weight	0.1
Optimiser	Adam
Learning rate	0.005
Weight decay	10^{-5}
Batch size	1000
Epochs per phase	20
Sequence passes	1
Seeds	25

The convolutional channel count and the absence of a hidden bottleneck reported above are examined in the architecture ablation of Section S.8.

3.1.4 Results

Table 2: Continual-learning results for the champion configuration of Table 1, $n = 25$ seeds.

Metric	Value ($n = 25$ seeds)
AA	0.9087 ± 0.0051
BWT	-0.0501 ± 0.0059
Class-IL	0.9101 ± 0.0050

Across 25 seeds, the champion configuration reaches an average per-task accuracy (AA) of 0.9087 ± 0.0051 . After all five phases, each digit pair’s held-out test accuracy averages just above 90%. Backward transfer (BWT) is -0.0501 ± 0.0059 , a small negative number showing that, on average, a task’s own accuracy drops by about five points between when it is first learned and the end of training, the residual forgetting predicted and bounded by the interference bound described later. The pooled Class-IL accuracy, 0.9101 ± 0.0050 , is close to but not identical with AA; it confirms the same result under the harder, single-head, ten-way evaluation used throughout the later literature comparison.

Figure 5 unpacks this aggregate picture into the per-task-pair result matrix R it is computed from, for an independent 25-seed confirmation run of the champion configuration (AA = 0.9061 ± 0.0065 , BWT = -0.0521 ± 0.0074 , within noise of Table 2’s headline numbers). Panel (a) shows the full mean R matrix as a heatmap in which each row is the accuracy of every task immediately after a given phase, so the lower triangle traces how earlier tasks’ accuracy erodes as later phases are trained. The entries directly above the diagonal are a structural zero, not a measured accuracy. Panel (b) isolates each task’s own accuracy trajectory, from the phase it is first learned to the end of training. Every task declines from its own peak, at different rates, with digit pair $\{4, 5\}$ showing the steepest drop of the five, a task in the middle of the five-phase sequence rather than the one recency alone would predict as worst; digit 4’s specific confusability with digit 9, examined later, is a more likely driver of this particular task’s decline than its position in the training order.

Figure 6 shows the trained model itself at the end of training, for one seed (seed 1007) run under the same champion hyperparameters (Table 1). It gives the decoded prototype $w_j = \text{dec}(e_j)$ at every one of the 625 grid units and the per-unit maturity trace c_{mat} (Section 2.3) each unit ends training with. Panel (a) shows the topographic organization Topographic Gating produces directly. Each digit occupies a contiguous region of the grid centered on its final anchor, and the region’s own prototypes smoothly interpolate that digit’s stroke variations toward its neighbors, the queryable, everywhere-meaningful space Section 1.1 argues for, in contrast to a set of isolated stored points. Panel (b) makes visible the geometric mechanism behind Section 1.3’s protection argument. Maturity is high in a compact island around each anchor and falls to near zero away from it, so the narrowed neighborhood bandwidth that high maturity buys (Section 2.3) is itself spatially confined to the region a class actually occupies. The single confirmed champion run (1 pass over the five-phase schedule, as reported throughout this section) would show no island at all for digits 8 and 9, because maturity for a class only updates when that class is later replayed while training a subsequent phase (Section 3.1.2); under a single pass no phase follows the last one, so the two final classes never receive that update, exactly the structural asymmetry a later corollary predicts. Figure 6 instead uses the same seed and hyperparameters run for two full passes over the schedule (replays = 2; AA = 0.918, BWT = -0.037 for this run, distinct from the single-pass configuration Table 2 reports), so as to show every class’s maturity island instead of exhibiting that asymmetry a second time. The second pass’s own phase 1 replays every class from the first pass, including digits 8 and 9, giving them the one maturity update a single pass structurally withholds.

The protocol and hyperparameters above are run under Topographic Gating’s geometric

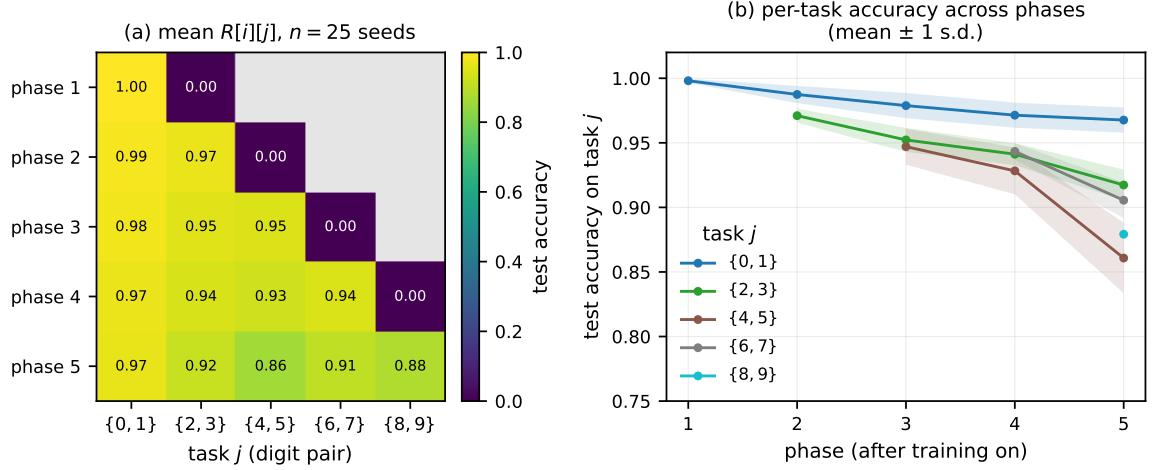


Figure 5: Continual-learning results matrix for the champion configuration (Table 1), $n = 25$ seeds. (a) Mean $R[i][j]$: task j ’s test accuracy immediately after training phase i ; the upper triangle beyond the first superdiagonal is never measured (grey), and the first superdiagonal is the structural pre-training zero. (b) Each task’s own accuracy across phases from the phase it is first trained onward (mean $\pm$ one standard deviation over seeds), the trajectory Backward Transfer (BWT) summarizes into a single number.

protection, that a converged class’s narrowed gate does not reach another class’s target position (Section 1.3). That property, developed further below, becomes an interference bound on prototype drift, derived from the same STM kernel and confirmed against the results reported here.

3.2 Interference in Topographic Gating

3.2.1 The learning-leakage bound

During a phase that trains only one class c , prototypes belonging to other, already-learned classes still receive nonzero updates, because the STM kernel is soft rather than a hard winner-take-all, so every unit receives some pull on every step, including units nowhere near the class currently being trained. That residual pull is the mechanism of catastrophic forgetting in this architecture. It is identical whether w_j is the shallow model’s own stored row or the shared decoder’s gated first layer, since both are driven by the same $\varphi_{i,j}^{\text{STM}}$ kernel (Section 1.2.4); only what lies downstream of that gated layer differs between the two. Model the online update $w_j \leftarrow w_j + \eta \varphi_{i,j}^{\text{STM}} (x_i - w_j)$ as a leaky integrator. Because $\varphi_{i,j}^{\text{STM}}$ ’s two Gaussian factors (neighborhood and anchor) fuse into a single Gaussian of fixed width σ_{eff} centered at a coincidence-weighted point between the best-matching unit and the anchor, this gives a bound on how far any prototype can drift during a phase. The bound decays doubly with grid distance to the anchor, a Gaussian nested inside an exponential, and falls to negligible beyond a few σ_{eff} . In quantitative form, this bound says that anchor placement alone, with no extra protective mechanism, already suppresses cross-class interference as a geometric consequence of kernel shape. The bound holds under realistic best-matching-unit scatter around the anchor, not only the idealized case where the two coincide exactly, since a scatter radius ρ shifts the effective origin of the decay without removing it. It also depends on the *shape* of the neighborhood-bandwidth annealing schedule within a phase, not just its start and end values. Front-loaded schedules accumulate less learning leakage than back-loaded ones, matching an independently observed empirical effect that the bound was not designed to explain. This is stated formally later, as a lemma and a proposition, each with

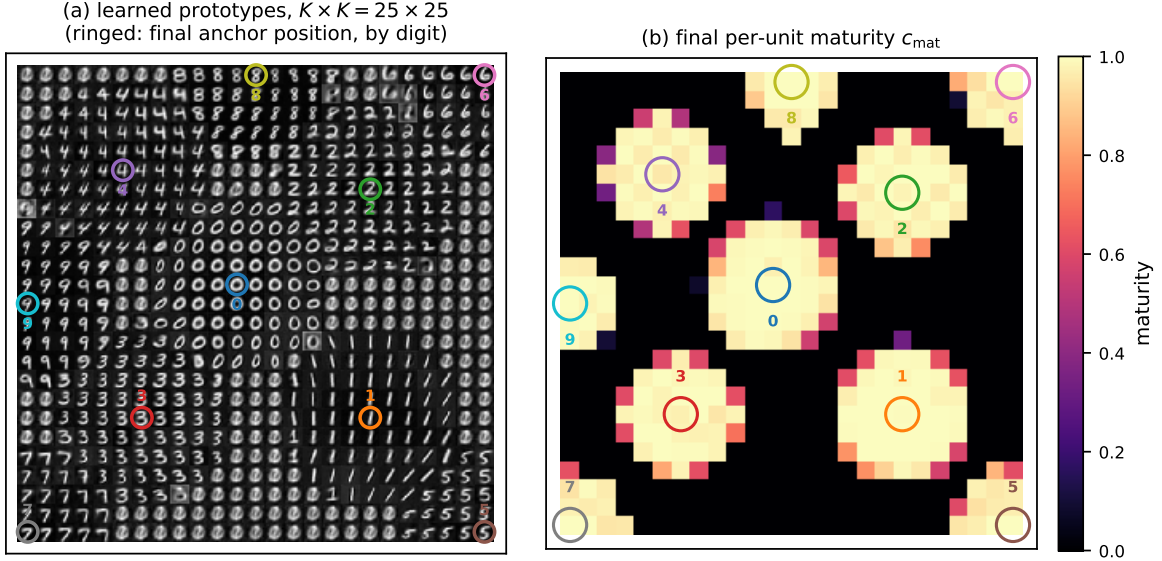


Figure 6: The trained model at the end of training, seed 1007 under the champion hyperparameters of Table 1 but run for two full passes over the five-phase schedule (replays = 2, distinct from the single-pass configuration used elsewhere in the paper) so that every class, including the two classes trained last, receives at least one post-training maturity update (see text). (a) Decoded prototype $w_j = \text{dec}(e_j)$ for every grid unit j , assembled into a single $K \times K$ mosaic of 28×28 tiles (each independently contrast-normalized); rings mark each digit’s final anchor position. (b) Final per-unit maturity trace c_{mat} (Section 2.3) over the same grid, same anchor rings.

corollaries and proofs.

This section and a later one bound learning leakage and generative replay’s refresh term separately. Neither says how much of Section 3.1.4’s observed anti-forgetting comes from Topographic Gating’s own geometric protection versus generative replay’s refresh term. The experiment below separates the two empirically.

3.2.2 Experimental setting

We compare the confirmed configuration of Section 3.1.3 against an otherwise identical one with generative replay disabled, both evaluated at 25 seeds, and report average accuracy (AA) and backward transfer (BWT).

3.2.3 Results

Average accuracy (AA) and backward transfer (BWT) are:

Configuration	AA	BWT
Replay enabled (confirmed)	0.9087 ± 0.0051	-0.0501 ± 0.0059
Replay disabled (ablation)	0.9002 ± 0.0089	-0.0554 ± 0.0087

The gap is small in absolute terms (0.0086 on AA, 0.0053 on BWT), far short of the near-chance collapse a mechanism with no anti-forgetting device at all shows on this protocol. A Welch two-sample comparison nonetheless finds it statistically reliable at this sample size ($t = 4.11$, $df \approx 38$ for AA; $t = 2.48$, $df \approx 42$ for BWT), with both configurations run on the same 25-seed protocol used elsewhere (Section 3.1.3).

Two results follow from this comparison. Topographic Gating alone accounts for the large majority of anti-forgetting behavior observed with replay enabled: disabling replay costs under one point of AA, far from the catastrophic collapse a purely parameter-importance or purely exemplar-free method shows without any protective device. Replay itself makes a modest but measurable contribution, partially correcting, per the two-term account developed later, the residual forgetting Topographic Gating cannot explain. A BWT of -0.0554 with replay disabled remains clearly negative, consistent with a non-topographic learning-leakage pathway operating regardless of replay; replay narrows that residual by only about a tenth (0.0053 of 0.0554).

The bound above quantifies how far a prototype drifts and how much of that drift replay corrects. It says nothing about whether a given amount of drift is enough to turn one class into another at test time. The next section addresses that question on a dataset chosen for a case the bound above is silent on. There, a group of classes sits close enough together, before any training-induced drift at all, that the same drift a well-separated class shrugs off is enough to cause real confusion.

3.3 A Capacity Limit on the Interference Bound

The interference bound above treats every pair of classes alike, since learning leakage decays with grid distance regardless of which two classes happen to sit at that distance. That bound leaves open whether two classes’ representations are close enough, independent of any training dynamics, for ordinary learning leakage to cause a real classification error.

3.3.1 The confusable-cluster diagnosis

We ran this diagnosis directly on trained models that already existed: the MNIST models from Section 3.1.4. We also trained new models on the FashionMNIST champion configuration. For FashionMNIST, we tuned the hyperparameters independently instead of inheriting them from Section 3.1.3: anchor bandwidth $\sigma_t = 1.0$ and neighborhood schedule floor $\sigma_r^{\text{final}} = 0.6$. All other settings are unchanged. We computed confusion matrices directly from these models, pooled across every confirmed seed on record for each configuration (45 for MNIST, 5 for FashionMNIST), instead of inferring class-level structure from the aggregate AA/BWT summary.

This exposes a sharp qualitative difference between the two datasets in how learning differs from class to class. On MNIST, digit 4’s single largest confusion is digit 9, at 9.9% of true-4 test images (4372/44190) — the single largest off-diagonal entry anywhere in the matrix (Figure 7a). Confusion elsewhere is smaller but not negligible: digit 5’s largest confusion is with digit 3 (3082/40140, 7.7%), and digit 7’s largest is with digit 9 (2881/46260, 6.2%), both below the digit-4/9 peak without approaching MNIST’s diffuse-cluster territory. FashionMNIST looks different. Pullover is misclassified as Coat or Shirt a combined 36.9% of the time (1152/5000 to Coat, 694/5000 to Shirt), and the confusion is bidirectional (Coat→Pullover 25.6%, Shirt→Pullover 22.1%, with Shirt additionally confusable with T-shirt (21.9%) and Coat (18.5%)). Four classes, T-shirt, Pullover, Coat, and Shirt, form a mutually confusable cluster (Figure 7b) with no comparable structure among MNIST’s ten digits. The cluster’s own accuracy is 0.5248, against 0.8354 for FashionMNIST’s remaining six classes, itself close to MNIST’s own 0.9101. FashionMNIST’s lower aggregate performance throughout this section is not evidence of a uniformly harder dataset. It is concentrated almost entirely in these four classes: within each of the two task pairs that contains a cluster member, that member (Pullover, Coat) absorbs nearly all of it, while its task-mate (Dress, Sandal) does not. We call such a group of classes a confusable cluster, and define cluster confusability as the degree to which its members are mistaken for one another, in the form of pairwise misclassification rates that are high and mutual, well above what any comparably-sized grouping of classes shows elsewhere in the same dataset, so that the group’s own accuracy sits far below the rest of the dataset’s. This diagnosis indicates that forgetting under this protocol concentrates in whichever specific classes happen to sit closest together in the

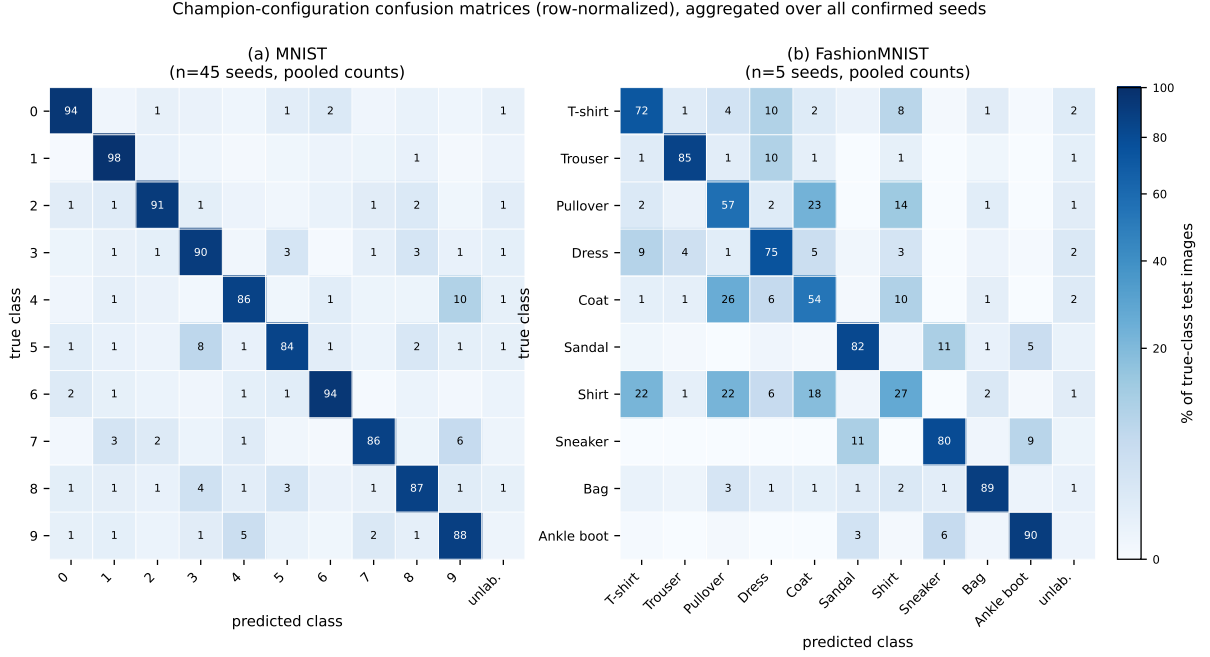


Figure 7: Champion-configuration confusion matrices, row-normalized (percentage of each true class’s test images), pooled across every confirmed seed on record: (a) MNIST, $n = 45$ seeds; (b) FashionMNIST (anchor bandwidth $\sigma_t = 1.0$, neighborhood schedule floor $\sigma_r^{\text{final}} = 0.6$), $n = 5$ seeds. “unlab.” is the fraction of test images whose best-matching unit carries no majority-vote class label.

model’s own representation, whatever the dataset, while every other class stays largely intact. It is a property of class geometry. In the following section, we examine the factors that determine this closeness and assess the extent to which it can be reduced.

3.3.2 Hypothesis: training dynamics

Hypothesis. The cluster’s confusability could be a training-dynamics problem, where the loss might shape updates in a way that fails to separate these four classes’ prototypes, or units might be protected prematurely once they reconstruct a confusable sample well. If so, a mechanism that reshapes the training signal around this failure mode should reduce it.

Experimental setting. We tested four candidate mechanisms in sequence. The first two act on which updates a unit is allowed to receive; the last two act on the loss itself.

Classification-gated maturity restricts the per-node maturity trace (Section 2.3), which protects a well-learned unit’s neighborhood from further disruption once it advances. Ordinarily, the trace advances whenever a replayed sample reconstructs well at its unit, regardless of whether the unit’s class prediction is correct. The gate makes it advance only when the replayed sample’s nearest-anchor prediction also matches its true label, so maturity reflects classification competence rather than reconstruction fit alone. This targets a specific failure mode: a unit serving a confused class can reconstruct its samples well while still consistently mispredicting their class, and would otherwise earn the same protection as a unit that is actually working, entrenching the confusion instead of correcting it. It has no tunable parameter of its own.

The *neighbor-restricted* variant narrows the same gate further. As stated, it withholds maturity on any misprediction, including ones to classes with nothing in common with the true class. This variant withholds maturity only when the predicted class is among the true

class’s k nearest classes by pixel-space class-mean distance, computed once from the training set, concentrating the correction on classes that are actually competing for the same representational territory rather than on any error at all. We swept $k = 3$, the smallest value that groups T-shirt, Pullover, Coat, and Shirt into one mutual neighborhood, confirmed directly against the class-mean distance matrix before sweeping.

Prototype-margin loss acts on what the decoder produces rather than on which updates a unit receives. For every pair of currently visible classes in the same $k = 3$ neighborhood, a squared-hinge penalty $\text{relu}(m - \|w_{r_i} - w_{r_j}\|)^2$, where w_{r_i} is class i ’s anchor unit’s decoded prototype and m the target margin, is added to the training loss whenever the two classes’ prototypes sit closer than $m = 8.0$ in pixel space, pushing them apart. A first calibration run at loss weight 0.05 left the hardest pairs unseparated, the added term too weak against the existing reconstruction and STM-energy terms; raising the weight sixfold, to 0.3, with the margin unchanged, separated every checked pair, and this became the swept configuration.

Edge-weighted loss targets a different bias in the same reconstruction and STM-energy terms: flat, low-information pixels dominate plain pixel-space distance, while the pixels that actually distinguish confusable classes sit on contours, for FashionMNIST’s confusable cluster for example, collar, sleeve, hem. A per-sample Sobel gradient-magnitude map of the target image reweights both terms so contour pixels count more than flat regions; best-matching-unit selection itself is left untouched. A calibration run at gradient weight 2.0 destabilized Sandal, whose thin-strap silhouette concentrates the reweighting onto very few pixels; halving the weight to 1.0 trained stably and became the swept configuration.

Results. Classification-gated maturity is null ($n = 20$: AA = 0.7192 ± 0.0084 vs. 0.7186 ± 0.0074), and the neighbor-restricted variant is null as well ($n = 20$), with the restriction independently verified to isolate the correct four-class cluster. The prototype-margin loss is zero-sum ($n = 5$: Shirt +0.033, Pullover −0.081), and the edge-weighted loss is likewise zero-sum ($n = 5$: Coat +0.060, Shirt +0.033, Pullover −0.081), with a net aggregate that is negative.

None of the four mechanisms raises the cluster’s own accuracy. Where a mechanism visibly changes anything, it *redistributes* accuracy among the cluster’s members instead of lifting them together: the prototype-margin loss and the edge-weighted loss both trade Pullover’s accuracy for Shirt’s (and, for the edge-weighted case, Coat’s) by an amount close to what Pullover loses. This pattern is more consistent with the four classes competing for a fixed representational budget than with any one mechanism failing to find the right update rule.

Applied instead to MNIST’s isolated 4/9 pair, the classification-gated maturity mechanism produces a real, replicated per-class improvement: digit 4’s accuracy rises by 0.0065 and digit 9’s by 0.0212 (both $n = 5$), with the variance across seeds roughly halved for both digits relative to the unmodified baseline. The same mechanism that is null on the four-way cluster succeeds reliably on the two-way pair. The two cases differ only in the geometry the mechanism is asked to fix.

3.3.3 Hypothesis: routing

Hypothesis. The cluster’s confusability could be a routing problem. Best-matching-unit selection (Section 2.1) is computed against plain pixel distance, which, for FashionMNIST’s confusable cluster for example, is dominated by overall garment silhouette; confusable images may be assigned to the same or neighboring units at this stage regardless of what those units subsequently learn to draw.

Experimental setting. We tested two variants of edge-aware routing, each on a single calibration seed. Both replace plain pixel distance with an edge-augmented representation at every point routing is computed; they differ only in whether the edge channel is derived from the same decoder as the pixel channel or given its own.

Shared-pathway routing makes best-matching-unit selection itself edge-aware. Every distance computation used for routing, best-matching-unit selection, STM energy, and local STM energy, compares the real image and each prototype as a concatenation of raw pixels and a Sobel gradient-magnitude map of that same output, computed by one shared function applied identically to both, rather than as plain pixels alone. Reconstruction and replay still use the decoder’s plain-pixel output; only the representation used for comparison changes. Using one function for this comparison everywhere it is needed, rather than separate code paths for training and evaluation, guarantees that training-time and evaluation-time routing agree by construction, not by convention: every recognition method (Section 3.1.1) inherits the edge-augmented comparison automatically, through the same call that already computes best-matching-unit selection. It has no tunable parameter.

Split-pathway routing instead gives the edge channel its own decoder, independently parameterized from the one producing the pixel channel, so the two channels no longer share weights. Deriving the edge map mechanically from the pixel decoder’s own output, as the shared-pathway variant does, ties the edge channel to whatever the pixel decoder already draws; an independent pathway can instead specialize on the edge representation directly, unconstrained by the pixel channel’s own optimization.

Results. On FashionMNIST, a single calibration seed showed T-shirt, Pullover, and Coat all improving simultaneously, the only mechanism in this investigation to move three cluster members in the same direction at once. The result did not survive confirmation at $n = 5$. Aggregate accuracy fell below baseline ($AA = 0.7011 \pm 0.0131$ against 0.7186 ± 0.0074 ; $BWT = -0.1476 \pm 0.0160$ against -0.1408 ± 0.0093) with roughly twice the baseline’s seed-to-seed variance, and Pullover’s own improvement reversed to a loss once averaged. A second variant gave the edge channel its own independently-parameterized decoder pathway instead of deriving it from the same pathway that also has to produce a plausible image; it performed worse still at a single calibration seed than the simpler version it was meant to improve on. Edge-aware routing is a reliably noisier training signal in this architecture, not a fix waiting for the right implementation, and the instability does not track with how much capacity the edge channel is given.

3.3.4 Hypothesis: capacity

Hypothesis. The cluster’s confusability could instead be a capacity limit rather than a training-dynamics or routing problem. Every prototype in the shared convolutional decoder is computed as $w_j = \text{dec}(e_j)$ (Section 2.1), the same set of shared weights mapping every one-hot grid position through convolutional layers to pixel space. If those shared weights lack enough independent degrees of freedom to represent four visually similar classes as distinct outputs, no amount of retraining that shared decoder, gating which units it updates, or reshaping the loss it is trained against, as tested above, can separate them: the ceiling on what the decoder can express, not how it is trained, would be the limiting factor. If so, adding representational capacity, rather than reshaping how the existing capacity is used, should help where the training-dynamics and routing mechanisms above did not.

Experimental setting. The architecture already contains a natural instrument for this test: the shallow baseline’s own prototype table (Section 2.1, Figure 3), a directly-addressed (K^2, D) matrix with an independent row per grid unit and no parameters shared across units, the structural opposite of the convolutional decoder’s shared weights. *Parallel-shallow capacity* adds this table as a second, independently-parameterized pathway summed with the convolutional decoder’s output, so the decoder’s forward pass becomes $\text{dec}(z) = \text{dec}_{\text{conv}}(z) + \text{dec}_{\text{shallow}}(z)$, with no other change to the training loop (Figure 8): every unit can now draw on capacity that is not shared with any other unit, in addition to the convolutional pathway’s shape-sharing capacity.

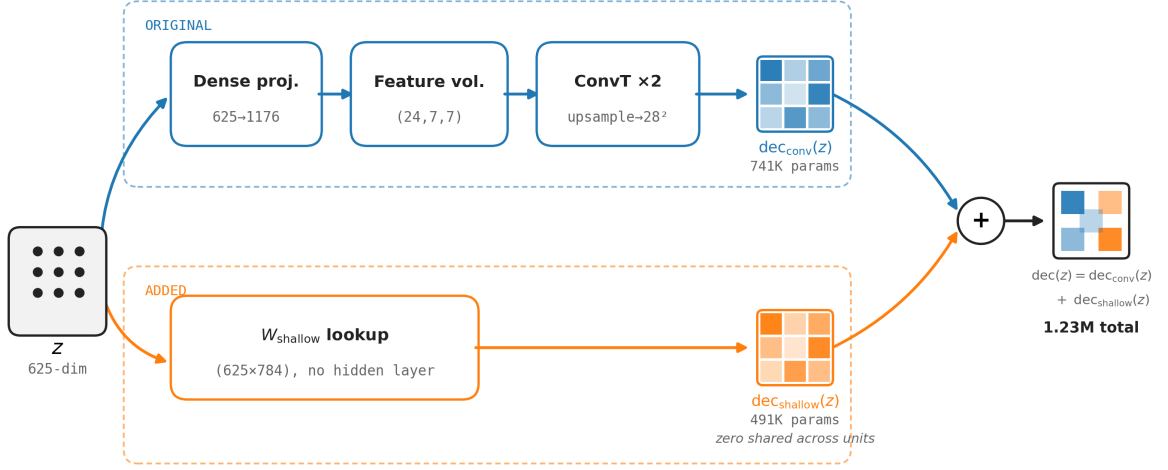


Figure 8: The capacity fix sums a second, independently-parameterized pathway onto the decoder of Figure 4: the existing convolutional decoder (dec_{conv} , blue, 741K params, unchanged from Figure 4) and the added shallow prototype table ($\text{dec}_{\text{shallow}}$, orange, 491K params, no parameter shared across grid units), summed to 1.23M total.

Three controls isolate what specifically produces the FashionMNIST gain, instead of leaving it as an unexplained correlation with adding a second pathway. *Bottleneck rescue* reintroduces the decoder-width bottleneck ($h \in \{32, 64, 128, 256\}$) alongside the added shallow pathway, to check whether the shallow pathway is merely compensating for an under-provisioned convolutional stage rather than adding capacity of its own. *Shallow pathway alone* removes the convolutional decoder entirely, leaving only the added shallow-style table, to check whether the convolutional pathway was doing any real work of its own that the unconstrained table cannot replicate. *Two convolutional pathways* replaces the shallow pathway with a second, independently-parameterized convolutional decoder of the same architecture (disjoint weights), summed identically, to check whether any second pathway would have helped regardless of its lack of shared parameters, isolating a generic redundancy or ensemble effect from the shallow table’s specific unconstrained, per-unit structure.

Results. The improvement is small but confirmed at $n = 20$ on FashionMNIST ($\text{AA} = 0.7226 \pm 0.0094$ against 0.7186 ± 0.0074 , $\Delta = +0.0040$; $\text{BWT} = -0.1336 \pm 0.0089$ against -0.1408 ± 0.0093 , $\Delta = +0.0072$). The gain does not land on the confusable cluster itself, which returns to within noise of its unmodified accuracy; it lands on two other classes (Trouser, Dress) that were not the mechanism’s original target. On MNIST, the identical change makes no measurable difference ($\text{AA} = 0.9073 \pm 0.0051$, $n = 19$ excluding one severe single-seed outlier, against 0.9089 ± 0.0053), consistent with a capacity account in which MNIST’s digit classes are already well separated by the unmodified decoder and have little capacity pressure left to relieve. Bottleneck rescue reduces the loss to 1 to 7 points (versus ~ 10 points alone) but does not eliminate it; the unbottlenecked configuration ($h = 0$) remains best. Shallow pathway alone performs substantially worse (4 to 6 points), with chaotic, non-uniform per-class swings and two simultaneously destabilized classes by the diagnostic of Section 2.3. Two convolutional pathways gives no benefit at all ($+0.0005$ against baseline at the same seed, against the shallow pathway’s own $+0.022$ at that same seed).

The third control matters most: it rules out the possibility that any second, independently-parameterized decoder path helps through some generic redundancy or ensemble effect. Two convolutional pathways share the same representational constraints (the same convolution kernels

reused at every spatial position, the same inability to give any one class a fully independent degree of freedom) and, summed together, gain nothing over one. The property doing the work is specifically the shallow pathway’s lack of any shared parameter across grid units, the same property that gives the shallow baseline its exact zero-forgetting guarantee in Section 1.2.4. The convolutional decoder structurally lacks this property, and adding it back in, even partially, relieves the bottleneck. The first control shows this capacity gain is not merely compensating for an under-provisioned convolutional stage. Even with the convolutional bottleneck reinstated, the shallow pathway only partially rescues the loss, confirming the two pathways are not simply substitutable for each other. The second control shows the convolutional pathway does real representational work of its own: removing it costs more than adding the shallow pathway gains.

3.3.5 Synthesis

The three hypotheses tested above narrow the explanation for confusable-class forgetting by elimination, not by direct measurement. Four ways of reshaping the training signal around the cluster fail, including one that is a confirmed, replicated success on an easier, unconfusable pair; this rules out the mechanism itself being broken. Two ways of changing routing fail, one worse than the other despite having more capacity; this rules out better assignment alone fixing it. One way of adding capacity, isolated by a three-way control against the alternative explanations of “any second pathway” and “the convolutional stage was merely under-provisioned,” yields a small but genuine improvement specifically where capacity pressure plausibly exists (FashionMNIST), and no improvement where it does not (MNIST).

Section 3.2’s bound treats the boundary spacing between confusable classes’ representations, how much of the decoder’s fixed capacity separates them before any training-induced drift at all, as a fixed background condition. The evidence above suggests that spacing is set by capacity, and not by anything the bound’s training dynamics could adjust. That connection is the natural bridge between the two sections. This account is the most consistent reading of the evidence available, but it is inferred rather than directly measured, since nothing here measures decoder representation-space distance between confusable and non-confusable class pairs, with or without the added shallow pathway. Doing so is a natural next test of the hypothesis, taken up in Future Work.

4 Empirical Comparison to the Continual Learning Literature

This section situates the champion configuration’s accuracy against the continual-learning literature: first on Split-MNIST and Split-FashionMNIST directly, then against the architecturally closest peer group of SOM- and prototype-based methods, then against results published after the 2019/2020 baseline literature this paper’s own protocol was built against. A final subsection lists the caveats that qualify every comparison above it.

4.1 Scenario and metric

Van de Ven and Tolias (van de Ven and Tolias, 2019) distinguish three continual-learning scenarios by what is available at test time: Task-IL provides task identity and a per-task output head; Domain-IL withholds task identity but keeps a single shared binary head; Class-IL withholds task identity and requires a single head spanning every class seen so far. This paper reports Class-IL because it is the harder and more realistic of the three. No task identity is available at test time, so a test image’s label depends entirely on the model’s own ten-way majority-vote best-matching-unit labeling (Section 3.1.1), built over every class trained so far. Methods evaluated only in Task-IL or Domain-IL are noted as such below and kept out of the Class-IL column. Without rehearsal or parameter isolation, Class-IL collapses most regularization methods to near-chance accuracy, since the shared output head is overwritten by each new task.

4.2 Split-MNIST

Table 3 reproduces the standard Class-IL comparison for Split-MNIST from (van de Ven and Tolias, 2019), the reference this project’s own protocol was built against, alongside this paper’s champion configuration (Section 3.1.3).

Table 3: Class-IL accuracy on Split-MNIST. Literature values from (van de Ven and Tolias, 2019); this paper’s own result is the pooled Class-IL metric, $n = 25$ seeds. Params: $\dagger$ computed from the reported 784-400-400 backbone shared by these baselines (not separately stated per method in the source); $\ddagger$ computed from (van de Ven and Tolias, 2018)’s explicit architecture specification for the same fair-comparison MLP convention: a 784-400-400-10 classifier plus a separate symmetric VAE generator (encoder/decoder each 400-400 hidden, latent size 100).

Method	Class-IL Acc.	Params	Category
EWC (Kirkpatrick et al., 2017)	20.0%	479K $\dagger$	regularization
SI (Zenke et al., 2017)	20.0%	479K $\dagger$	regularization
LwF (Li and Hoiem, 2017)	23.9%	479K $\dagger$	knowledge distillation
DGR (Shin et al., 2017)	90.8%	1.55M $\ddagger$	generative replay
Topographic Gating (ours, champion)	91.0 $\pm$ 0.5%	741K	topographic anchor segregation + generative replay
iCaRL (Rebuffi et al., 2017) (2000 ex)	94.6%	479K $\dagger$	exemplar + nearest-class-mean
Joint training (upper bound)	97.9%	479K $\dagger$	oracle

The champion configuration sits above every no-replay baseline and above DGR, the closest generative-replay comparator with a similarly exemplar-free footprint, while remaining below iCaRL’s largest-buffer variant, which stores 2000 real exemplars, something this method does not do at all.

4.3 Split-FashionMNIST

Table 4 reproduces the Fashion-MNIST results of Deja et al. (Deja et al., 2020), whose protocol (five tasks, no task identity, cumulative test set) matches this paper’s, alongside both FashionMNIST configurations reported in this paper: the champion configuration unmodified, and the capacity fix of Section 3.3.4.

Table 4: Average accuracy (AA) on Split-FashionMNIST. Literature values from (Deja et al., 2020), Table 2; this paper’s own results are AA, $n = 20$ seeds each; Table 3 instead uses the pooled Class-IL metric. Params: $\dagger$ exact, computed from (Deja et al., 2020)’s own released code (the shared LeNet classifier, $d = 32$: 3 conv layers of 32/64/128 filters, 4×4 kernel, FC-84, FC-10; no separate generator needed for these buffer-based methods); $\ddagger$ classifier only (same $\dagger$ count); this method also trains a separate generator whose exact size for this reimplementaion is not stated in the paper or its released code (only BinPlay’s own generator is); $\S$ not reported: this method’s classifier and generator form a single unified network by design, unlike the shared LeNet backbone, and no FashionMNIST-scale specification was found.

Method	AA	Params	Category
GEN-MIR	52.4%	$\geq 263\text{K}^\ddagger$	generative replay
iCaRL (Rebuffi et al., 2017)	67.7%	$263\text{K}^\dagger$	exemplar + nearest-class-mean
GR (Shin et al., 2017)	68.0%	$\geq 263\text{K}^\ddagger$	generative replay
ER-MIR	69.7%	$263\text{K}^\dagger$	episodic replay
OCDVAE	69.9%	$n/r^\S$	generative replay
ER	70.2%	$263\text{K}^\dagger$	episodic replay
GEM (Lopez-Paz and Ranzato, 2017)	70.3%	$263\text{K}^\dagger$	episodic replay
Topographic Gating (ours, champion)	$71.9 \pm 0.7\%$	741K	topographic anchor segregation + generative replay
Topographic Gating (ours, capacity fix)	$72.3 \pm 0.9\%$	1.23M	+ parallel shallow pathway, Section 3.3.4
RTF	75.2%	$n/r^\S$	generative replay
GR + distillation (Shin et al., 2017)	78.1%	$\geq 263\text{K}^\ddagger$	generative replay
BinPlay (Deja et al., 2020)	81.4%	$1.98\text{M}^\dagger$	generative replay

Both FashionMNIST configurations land in the middle of the field, above GEM, ER, ER-MIR, GR, and OCDVAE, and below the strongest generative-replay methods (RTF, GR plus distillation, BinPlay). Every method in this table scores lower on FashionMNIST than the corresponding Split-MNIST result reported in its own paper. This drop is common across architectures, consistent with FashionMNIST containing a genuinely confusable class cluster that MNIST does not (Section 3.3.1).

4.4 The closest peer group: SOM- and prototype-based methods

The methods architecturally closest to this paper’s own are built on self-organizing or prototype-based layers; none use a fully distributed representation. DendSOM (Pinitas et al., 2021) augments a layer of self-organizing maps with per-class hit matrices and reports outperforming EWC and SI substantially in Class-IL and Domain-IL, the same harder scenario reported here, while trailing both in the easier Task-IL setting. SatSOM (Urbanik and Gajewski, 2025) freezes individual units once they saturate, using no replay, and reaches 77.0% on FashionMNIST under a strict single-pass, one-class-per-phase protocol; that protocol is not directly comparable to the five-task class-pair split used throughout this paper. SOM plus VAE synthetic replay (Thapa et al., 2025) combines a self-organizing map with a variational generative model in a similar spirit to the generative replay used here, and reports results competitive with the replay-based methods in Table 3. None of these three peer methods reports a number on this paper’s own

protocol, so the comparison here is qualitative and not a single ranked table. The family as a whole occupies the same architectural niche as this paper: topology or prototypes in place of a fully distributed backbone, with anti-forgetting built directly into the representation’s geometry, not bolted on as a penalty term.

4.5 Beyond the 2019 and 2020 baseline literature

The comparisons above reproduce literature from around 2019 and 2020, close to when this project’s own evaluation protocol was designed. Within this same regime (no pretrained backbone, a comparable parameter budget, evaluated on Split-MNIST or Split-FashionMNIST), the field has not stood still. Brain-inspired replay (van de Ven et al., 2020), which replays context-modulated internal representations instead of pixels, reaches 91.5% on Split-MNIST in a later reproduction (Hu and Rostami, 2023). Adding a time-aware regularization term on top of that reproduction (Hu and Rostami, 2023) reaches 94.6%, a clear margin over this paper’s own 91.0% with no architecture change and no larger exemplar buffer. This is the closest published comparator to this paper’s own regime and result. A much larger literature reports substantially higher numbers on Class-Incremental benchmarks by relying on a large backbone pretrained before the continual stream ever starts. This paper’s decoder serves as the grounding mechanism the whole framework is built around (Section 1.1), not a classifier backbone; adopting a frozen pretrained backbone would abandon that motivating premise. For this reason, that literature is not treated as a comparison target here.

4.6 Comparability caveats

Four points qualify every comparison above. First, majority-vote best-matching-unit labeling is structurally closest to iCaRL’s nearest-class-mean classifier: both replace a softmax head with proximity to a stored representative. iCaRL’s representative is a learned feature-space mean; here it is a majority-vote label over a topographic unit, and no real exemplar is stored to compute it. Second, anti-forgetting in this paper comes from spatial segregation over a topographic substrate (Section 1.3); no buffer is stored. The comparison against no-replay baselines such as EWC and SI should therefore be read as a comparison of mechanisms, not merely of memory budgets. Third, the canonical Split-MNIST baselines in Table 3 use a two-layer multilayer perceptron backbone, and DendSOM and SatSOM are evaluated under different scenarios or protocols than the one used here; none of these architectural or protocol differences invalidate the comparisons above, but all should be kept in view: the numbers above are a ranking, not a like-for-like benchmark. Fourth, model size and per-step cost differ in ways the accuracy columns alone do not show. The shared 784-400-400 backbone behind the Split-MNIST baselines has approximately 479K parameters; this paper’s champion configuration has 741K, all in a single decoder network with no separate classifier (Table 3). Generative-replay methods add a second network on top of that shared backbone: DGR’s separate VAE generator (784-400-400-100 encoder and decoder, per (van de Ven and Tolias, 2018)’s architecture specification for the same fair-comparison convention) brings its total to 1.55M, more than double this paper’s champion, entirely because DGR trains two full networks where this paper’s replay reuses the one it already has. RtF (van de Ven and Tolias, 2018), which folds the classifier and generator into a single network with a shared encoder, needs only 1.07M for the combination, a direct, sourced illustration that architectural sharing of the kind this paper relies on for replay is also what the closest replay-architecture literature finds cheaper. For the Split-FashionMNIST table, only the shared LeNet classifier (263K, computed from (Deja et al., 2020)’s own released code) and this paper’s own configurations are precisely known; the generative-replay baselines there (GEN-MIR, GR, GR+distillation) are confirmed to add a separate generator on top of that 263K, but that generator’s exact size for this specific reimplement is not stated in the paper or its code, and OCDVAE and RTF’s unified architectures were not found at FashionMNIST scale, so their totals are left unreported;

estimating them by analogy across a different backbone and a different paper’s numbers would not be reliable. Per-step cost also differs in kind, not just in scale. GEM and A-GEM solve a gradient-projection problem against a stored episodic buffer at every step; iCaRL periodically recomputes class means over its stored exemplars; DGR trains and samples from its generator jointly with the classifier. This paper’s training step touches only the shared decoder and the grid’s φ^{STM} weights, with no stored buffer of any kind. The FashionMNIST capacity fix of Section 3.3.4 adds a second, independently parameterized decoder path (the shallow baseline’s own 491K-parameter prototype table) summed with the first, bringing that configuration to 1.23M parameters total (Table 4).

5 Discussion

5.1 Computational

5.1.1 Why topology, not just segregation.

Hard parameter-isolation methods such as PackNet (Mallya and Lazebnik, 2018) and HAT (Serra et al., 2018) already achieve zero forgetting through segregation, partitioning the network into disjoint, frozen per-task subsets with no topology involved at all. Topographic Gating could, in principle, be pushed toward the same regime by narrowing the neighborhood radius until it degenerates into disjoint masks. It is not, because a continuous substrate buys two things a disjoint one cannot. First, the interpolation and generative querying discussed in Section 1.2.3: any grid point decodes to a plausible sample, not only the points assigned a class, which makes generative replay and smooth reconstruction across class boundaries possible at all; a disjoint partition has no points other than the committed ones, so there is nothing to query in between. Second, a local, soft update rule of this kind is consistent with the topographic organization observed throughout cortex, where nearby neurons represent similar stimuli and updates propagate gradually across a neighborhood, in place of a mechanism with no known neural analogue. The price for both, taken up below, is a capacity that scales poorly with task count, a limitation inherited directly from the fixed grid size.

5.1.2 New theoretical contribution: a two-term interference bound.

The 2020 draft proves only that a single training step does not increase Topographic Gating’s own energy, for a fixed neighborhood width; it says nothing about how much a class is forgotten once training moves on. This paper adds that missing piece: a bound on forgetting itself.

The starting point is a simple fact about how the model updates its prototypes. Because the training signal is soft rather than winner-take-all, every training step nudges every unit a little, not just the unit currently being taught. When the model trains one class, the units storing an already-learned class still drift slightly, even though nothing about that class is being shown. We call this drift *learning leakage*.

Once a class’s units have settled near their assigned position on the grid, the two factors that shape a training step, one that spreads the update to nearby units and one that pulls it toward the class’s own assigned position, combine into a single, tightly localized region of influence. A unit’s exposure to leakage from another class then falls off doubly fast with its grid distance from that class’s position, a bell-shaped falloff nested inside an exponential one: units close to a class’s territory drift a little, units even a moderate distance away barely drift at all. In plain terms, simply placing classes at different positions on the grid, with no extra protective machinery, already suppresses interference between them, purely as a geometric consequence of how the update is shaped.

This bound extends from a single training step to a full training phase. Over a phase, the neighborhood radius is gradually narrowed, from wide to narrow, through annealing; the extended

bound shows that what matters is not just the radius’s starting and ending values but the *shape* of that narrowing over time. Narrowing the radius quickly, early in the phase, accumulates much less leakage than narrowing it slowly or mostly near the end, since the radius then spends less total time wide. This prediction was not designed to fit any specific result, yet it matches one: every test in this paper found that front-loaded narrowing schedules beat back-loaded ones.

The bound above covers a phase with no generative replay.

Replay complicates the picture: during a phase nominally training one class, a fraction of the training steps are decoder-generated images of previously seen classes, injected to keep them alive in memory.

We add a second term of the same mathematical form to capture this, a *refresh* term, which pulls an old class’s units back toward its own assigned position, opposing the leakage caused by training the current class. Net forgetting is then leakage minus refresh. Because the model’s replay budget is fixed and split evenly across every class seen so far, this two-term account makes a specific, testable prediction. The more classes accumulate, the smaller each old class’s own share of replay becomes, while that class’s exposure to leakage from every later phase keeps growing. Classes learned earlier should therefore forget more than classes learned later. The very last class trained in a run is a separate edge case: it receives no refresh at all, since no later phase exists to replay it back. Section 3.2.3 tests the two-term account directly.

5.1.3 Topological alignment: this paper’s place in the larger picture.

Topographic Gating instantiates the principle of topological alignment (Mannella and Tummolini, 2023; Zubek et al., 2025). Topological alignment is the hypothesis that an agent engaged in open-ended sensorimotor learning relies on multiple sensory and motor modalities, vision, touch, proprioception, and policy parameters, each instantiated as a distinct topographic map. Grounding this multimodal architecture requires these maps to become mutually consistent: stimuli arising from the same sensorimotor episode, though expressed in different modalities, must elicit corresponding activation patterns across their respective maps.

A shared anchor is the candidate mechanism for turning several independent topographic maps into one aligned picture. At each step, the currently selected policy’s own grid position becomes the anchor that every modality’s map is trained against. A single coordinate thereby comes to encode a triadic goal: a sensory condition, the policy that achieves it, and its sensory effect. Each of these is recoverable from that one coordinate by the same inverse, decoder-based mapping this paper builds and tests for one modality (Section 2.2).

Cross-modal correspondence is not an add-on to topographic organization; it follows directly from the same addressability and per-class protection this paper establishes for a single modality, applied once per modality and linked through the shared anchor. The atomic unit is the bimodal pair of one sensory modality and policy. The full multi-modality case is simply several such pairs composed onto one shared anchor (Zubek et al., 2025; Mannella and Tummolini, 2023).

5.2 Biological

5.2.1 Topographic organization is not a modeling convenience.

Functional topologies over sensory input features are widespread across vertebrate brains: cortical columns organize primary and associative sensory areas, and motor cortex shows an analogous organization by motor program. This is a general architectural principle, not an exception confined to one system or species (Kaas, 1997). A recent whole-brain synthesis extends the idea into a single topographic ontology spanning sensory, motor, and associative cortex alike (Arcaro and Livingstone, 2024). Cross-modal spatial registration of this kind is not purely learned. In the superior colliculus, where visual, somatosensory, and auditory space converge onto a shared map, multisensory spatial correspondence is already measurable at birth, before the extensive postnatal tuning that later sharpens it (Wallace and Stein, 2001). Topographic

circuits are typically laid down first by activity-independent guidance mechanisms, then refined by activity-dependent plasticity (Cline et al., 2023). The topographic principle also does not require discrete, modality-specific maps. Cortex-wide functional organization varies continuously along a small number of macroscale gradients, from unimodal sensory cortex to transmodal association cortex, not as a set of sharply bounded regions (Margulies et al., 2016; Huntenburg et al., 2018). Nor must the organizing dimension of a topographic map be a body surface or sensory epithelium. Numerosity is mapped topographically across human parietal cortex much as retinotopic position is mapped in visual cortex (Harvey et al., 2013). Natural-language semantic content tiles much of cerebral cortex in smoothly varying topographic gradients (Huth et al., 2016). And purely conceptual, non-perceptual relations between learned items are represented via a grid-like code structurally analogous to the one grid cells use for physical space (Constantinescu et al., 2016). This paper does not claim to model any of this circuitry. Its position is more modest: a continuous, low-dimensional topographic substrate is the same general representational format the brain reuses across sensory, motor, and more abstract domains alike.

5.2.2 Separation for protection, continuity for generalization.

The claim this paper actually depends on is not that the brain has topographic maps in general, as argued above. It is that the same spatial organization does two jobs at once, and the paper needs both.

Placing two classes' target positions far apart is the protective half. It names a broader computational strategy for reducing interference between representations: transforming similar inputs into more distinct, less overlapping codes so that they no longer compete for the same downstream resources. In memory and cerebellar circuits this strategy is usually called pattern separation, implemented by a recurring family of circuit motifs across strikingly different systems, from the mammalian cerebellum and dentate gyrus to the insect mushroom body (Cayco-Gajic and Silver, 2019). The mammalian hippocampal dentate gyrus is only the most extensively studied instance (Yassa and Stark, 2011), and even there the current state-of-the-art review treats the mechanism as unsettled: how much of the effect belongs to the dentate gyrus itself rather than upstream cortex, and what role adult neurogenesis plays in it, both remain open questions (Borzello et al., 2023). The same computation appears in circuits with no relation to the hippocampus at all: sparse, decorrelated expansion coding in the insect mushroom body improves learned odor discrimination through essentially the same logic (Lin et al., 2014). This separating strategy is itself spatially organized, not implemented uniformly by a single circuit. Within hippocampal area CA3, the balance between pattern separation and pattern completion varies continuously along the region's own transverse anatomical axis: more separation-like near the dentate gyrus, more completion-like farther away, not an all-or-nothing property of the region (Lee et al., 2015). Along the dentate gyrus's own dorsoventral axis, the same separating function is confined to a dorsal zone instead of spreading uniformly across the structure (Yeates et al., 2020). A comparable depth-wise gradient shows up in the cerebellar granule layer, a circuit unrelated to the hippocampus: granule cells nearer the white matter and nearer the Purkinje cell layer differ systematically in their biophysical properties and frequency preference, an arrangement that performs a Fourier-like decomposition of mossy-fiber input, expanding the population code and reducing overlap between input patterns (Straub et al., 2020). Sensory topographic maps carry the same strategy further: in whisker somatosensory cortex, local inhibitory dynamics sharpen and functionally segregate each whisker's barrel representation from its surrounding septal representation (Argunşah et al., 2026). Even an entire representational scheme can be organized this way: entorhinal grid cells cluster into a small number of anatomically discrete modules, each with its own characteristic scale, instead of varying continuously across the cortical sheet (Stensola et al., 2012), and the discreteness is functional, not incidental. A single spatial scale confuses distant locations that happen to fall at the same phase; keeping several scales in separate, non-interpolating modules lets the combined code resolve positions a single scale alone

would conflate, trading resolution for a large gain in how many locations can be told apart (Wei et al., 2015). Topographic Gating’s target-position placement does the same job spatially, over a continuous grid rather than a sparse code. The requirement in Section 1.3, that classes’ target positions keep sufficient mutual separation, is what makes the geometric protection argument hold, for anti-forgetting across sequential tasks rather than for interference-free storage within one.

Keeping the substrate continuous between committed positions, instead of partitioning it into disjoint regions, is the other half. It licenses the interpolation-based generalization the rest of the architecture depends on, from generative replay to reconstruction at coordinates never directly trained (Sections 1.2.3, 5.1.1). Biology offers precedent here too, both old and new. A population vector read out from directionally tuned motor cortical neurons generalizes smoothly to movement directions no single neuron is tuned to (Georgopoulos et al., 1986), a special case of the general result that smoothly tuned population codes support interpolation between the values they actually sample (Pouget et al., 2000). The same has now been shown at the level of population geometry itself, in more than one system: the joint activity of a grid-cell module lies on a continuous toroidal manifold, not a set of independently tunable units (Gardner et al., 2022), and the head-direction circuit’s population activity is confined to a comparably continuous ring-shaped manifold that persists, intact, from waking behavior into sleep (Chaudhuri et al., 2019). This continuity pays off exactly where this paper needs it to: hippocampal population activity encoding a continuously accumulated, task-abstract variable retains a smooth, low-dimensional geometric structure that generalizes to novel combinations of evidence never directly experienced during training (Nieh et al., 2021). Separation alone, implemented as a hard partition, would buy the protective half with none of this. What the grid in this paper relies on is the pairing of the two, not either on its own.

5.2.3 Scope of this claim.

The comparison stops well short of implementation: nothing above suggests the brain runs Topographic Gating, the STM kernel, or generative replay through a deep decoder in any literal sense. What carries over is narrower and easier to defend: the structural principle this paper relies on for anti-forgetting, protection through representational separation, has an independently documented biological precedent serving the same functional role of reducing interference between memories, and was not adopted merely to inflate a machine-learning benchmark score.

5.3 Computational limitations

Section 3.3 sharpens a limitation already implicit in a fixed $K \times K$ grid: task-count and class-count scaling is bounded by how much independent representational freedom the grid’s decoder can offer, and that freedom is exhausted first, and most visibly, by whichever classes happen to sit closest together in the representation the decoder already learns.

A first application of the confirmed architecture (Section 3.3.4’s capacity-added decoder, itself re-tuned) to Split-CIFAR-10 (natural, three-channel images, a substantially harder regime than either MNIST or FashionMNIST) reaches AA ≈ 0.26 after a coarse, single-pass hyperparameter screen, well below both datasets reported elsewhere in this paper. This is broadly in line with reported non-rehearsal, non-pretrained Class-Incremental Split-CIFAR-10 baselines in the wider literature (~ 24 to 31%), rather than a pathological result specific to this architecture, but it is not a finished result: only decoder channel count was found to help (confirmed at $n = 5$: AA = 0.2556 ± 0.0103 against 0.2451 ± 0.0064 at the inherited channel count, a real but modest gain traded against a genuinely worse BWT, -0.1549 ± 0.0071 against -0.1427 ± 0.0141), and several plausible levers (batch size, replay jitter, maturity-modulation constants, schedule shape) remain untested. We report it here as a limitation and a direction, not as a result this paper claims.

As stated in Section 1, this paper remains within a single modality. The cross-modal shared-anchor mechanism that motivates the broader “topological alignment” framework (Mannella and Tummolini, 2023; Zubek et al., 2025) is not exercised experimentally here.

5.4 Biological limitations

Section 5.2 argues for topographic organization as a general biological precedent, but leaves open a foundational disagreement about what a neural representation actually is. Barack and Krakauer (2021) distinguish two broad views of how the brain implements cognition. The Sherringtonian view treats cognition as operations carried out at the level of individual neurons and their specific wiring, so that a given piece of content is the business of a specific, locatable unit or small set of units. The Hopfieldian view instead treats cognition as trajectories and transformations within a population-level representational space, a state space or manifold whose geometry, not any single neuron’s identity, carries the computational content. Topographic Gating, as developed in this paper, sits uncomfortably close to the Sherringtonian pole: every class is assigned to a specific, addressable grid unit, and that unit’s own decoded prototype is treated as the class’s stored content, exactly the kind of locatable, unit-level code the Hopfieldian view was formulated to move away from.

The Sherringtonian pole has its own well-known liability, most concretely the grandmother-cell caricature: the idea, more often used as a foil than seriously defended, that a single neuron might respond exclusively and completely to one specific, richly structured concept, such as one’s own grandmother (Gross, 2002). Related, more modest single-neuron findings, such as human medial-temporal-lobe cells that respond selectively and invariantly to a specific familiar person across dramatically different views, and even to the person’s own name in text (Quiroga et al., 2005), are compatible with a highly sparse, highly localized code without settling the question, and the dominant view in current systems neuroscience leans toward the opposite pole: cognition, especially at the population level actually accessible to measurement, is carried by the joint, distributed activity of many neurons moving through a shared low-dimensional dynamical structure, not by the activity of any one of them in isolation (Vyas et al., 2020). Assigning each class a single addressable unit and prototype, as this paper’s grid does, risks the same objection: a critic could reasonably ask whether Topographic Gating merely relocates the grandmother-cell problem from a single biological neuron to a single artificial grid unit.

We do not think this tension is necessarily fatal, though nothing in this paper tests the following resolution directly. The grid unit a class is assigned to need not be the representation a downstream Hopfieldian-style computation reads out as its content; it can instead be the coordinate that configures a genuinely dynamical, population-level system elsewhere. Concretely, a grid position could parametrize the weights of a separate recurrent or dynamical module, in the manner of a hypernetwork, a network whose output is not a task answer but the parameters of another network (Ha et al., 2016), rather than parametrizing a static decoded output the way this paper’s own decoder does (Section 2.2). Under that reading, the addressable, low-dimensional grid this paper argues for is not itself the Hopfieldian representational space; it is a compact control space that selects which dynamics, among a continuous family, currently govern the system, an object closer to a Sherringtonian address book for picking out a regime than to a grandmother cell holding content of its own. Reconciling the two views this way, rather than choosing between them, is a direction for future work.

5.5 Future Work

Several concrete follow-ups remain open. Section 3.3.5 proposes, without measuring, that capacity narrows confusable-class forgetting by increasing the representation-space distance between confusable classes’ prototypes; the direct test is a pairwise-distance measurement between confusable and non-confusable class prototypes, with and without the added shallow

pathway of Section 3.3.4, checking whether the pathway’s accuracy gain tracks an increase in that distance rather than some other correlated change. The coarse hyperparameter screen reported in Section 5.3 for Split-CIFAR-10 remains incomplete: extending it to the remaining untested levers, and re-testing the capacity fix of Section 3.3.4 once a fuller CIFAR-10-specific configuration is established, rather than layering it onto an under-tuned baseline as done here, would complete that campaign. Section 3.2.3’s no-generative-replay ablation is MNIST-only; repeating it on FashionMNIST would test whether replay’s modest, non-topological correction generalizes to a dataset with a confusable cluster, where the refresh term’s dilution across competing classes may interact with cluster membership in a way the single-dataset result cannot show. Finally, the cross-modal shared-anchor mechanism (Section 5.3) is, at present, theory and prior work only; a multi-modal continual-learning benchmark exercising it directly is the natural extension beyond this paper’s single-modality scope.

6 Conclusion

This paper started from a simple claim (Section 1.1): a learned representational space is only as useful as what you can do with it after training ends, and that requires it to be addressable. Any coordinate should be meaningful and queryable, not just the ones training happened to visit. Topographic Gating provides that address book. We extend the single-layer supervised topological map of Mannella (2020) with a deep, shared decoder (Section 2.2). The addressing stays intact, and the decoder adds capacity a linear readout cannot offer. The price is a bounded, quantified leakage of learning across the shared layers below the topography (Section 1.2.4).

The rest of the paper’s results are consequences of that one addressing property. Protection against catastrophic forgetting falls out of target positions being placed apart on a fixed grid, each with a maturity-based, per-class notion of when it has converged, with no per-parameter importance estimate, no exemplar buffer, and no frozen partition to maintain (Section 1.3). Generative replay falls out of the same decoder already being continuously queryable, so rehearsal needs no second network to train or forget in its own right (Section 1.4). The two-term interference bound of Section 3.2 makes the first of these precise, decomposing forgetting into a learning-leakage term that decays doubly with grid distance and a refresh term contributed by replay, and predicting a recency asymmetry that remains untested but checkable directly against the bound.

Empirically, the champion configuration reaches $91.0 \pm 0.5\%$ Class-IL accuracy on Split-MNIST and $71.9 \pm 0.7\%$ average accuracy on Split-FashionMNIST (Section 4), matching or exceeding every exemplar-free baseline of comparable parameter count and falling short only of methods that store real exemplars or add a second regularization term the architecture studied here does not use. Where capacity, not the gating mechanism itself, is the binding constraint, the parallel shallow pathway of Section 3.3.4 recovers some of the shortfall without abandoning the shared decoder, and the preliminary Split-CIFAR-10 result (Section 5.3) shows the same architecture holding up, if not yet fully tuned, on a harder input distribution.

Section 3.3 names what remains open, and Section 5.5 spells it out. Capacity must keep scaling as the number of classes grows. And the shared-anchor mechanism at the core of topological alignment (Section 5.3), which ties maps of different modalities together, has never been tested beyond a single modality. That mechanism rests on the bimodal pair of stimulus and policy (Zubek et al., 2025; Mannella and Tummolini, 2023). What this paper delivers is one half of that pair, built on a controlled continual-learning benchmark: a topological map that is addressable, protected against forgetting, and open to generative queries. The cross-modal framework can now be built on top of it.

S Supplementary Material

S.1 The STM kernel and energy

Definition 1 (STM operator). *For input x_i with best-matching unit r_i and anchor (target grid position) $t_i \in Z$,*

$$\varphi_{i,j}^{\text{STM}} = \underbrace{\exp\left(-\frac{\|j - r_i\|^2}{2\sigma_r^2}\right)}_{\text{neighborhood term}} \cdot \underbrace{\exp\left(-\frac{\|j - t_i\|^2}{2\sigma_t^2}\right)}_{\text{anchor term}},$$

where σ_r (annealed over training) is the neighborhood bandwidth and σ_t (fixed throughout training) is the anchor bandwidth.

The neighborhood term is exactly SOM’s: it diffuses the update from the winner to its geometric neighbors. The anchor term is new: it suppresses the update for any node far from the class’s assigned target, regardless of how close that node is to the winner. Two limiting cases recover the simpler members of the family: $\sigma_t \rightarrow \infty$ removes the anchor term entirely and STM reduces to SOM; $\sigma_r \rightarrow 0$ collapses the neighborhood term onto a single winner and STM reduces to a label-modulated k-means.

The corresponding energy,

$$E(W) = \frac{1}{2} \sum_i \sum_j \varphi_{i,j}^{\text{STM}}(W) \|x_i - w_j\|^2,$$

is non-convex and non-smooth in W as a whole (the BMU $r_i(W)$ is piecewise-constant in W), but is strictly convex in W for any fixed assignment r , and is bounded below by zero.

S.2 Kernel fusion

Quantifying that residual pull (Section 3.2) requires two ingredients. First, the online update rule

$$w_j \leftarrow w_j + \eta \varphi_{i,j}^{\text{STM}}(x_i - w_j)$$

is a leaky integrator: each step moves w_j a fraction $\eta \varphi_{i,j}^{\text{STM}}$ of the way toward the current sample, so that after T steps at a roughly constant gain, w_j has moved a fraction $1 - \exp(-T\eta \varphi_{i,j}^{\text{STM}})$ of the way toward the training data, never more, since no single step can move w_j further than $\text{diam}(X)$. Second, that gain is only “roughly constant” if $\varphi_{i,j}^{\text{STM}}$ itself does not depend on which particular sample i is currently being trained, which kernel fusion below establishes: once training has settled, $\varphi_{i,j}^{\text{STM}}$ collapses to a single fixed function of grid distance alone. Substituting this fixed gain into the leaky-integrator convergence formula yields the interference bound derived next. The bound decays doubly, as a Gaussian nested inside an exponential. It quantifies this paper’s central claim: anchor placement alone, without any additional protective mechanism, suppresses cross-class interference as a geometric consequence of the kernel’s shape.

Lemma 1 (Kernel fusion). *For any best-matching unit r_i and anchor t_i (not necessarily equal), the STM kernel $\varphi_{i,j}^{\text{STM}} = \exp(-\|j - r_i\|^2/2\sigma_r^2) \cdot \exp(-\|j - t_i\|^2/2\sigma_t^2)$ factorizes as*

$$\varphi_{i,j}^{\text{STM}} = \underbrace{\exp\left(-\frac{\|r_i - t_i\|^2}{2(\sigma_r^2 + \sigma_t^2)}\right)}_{\text{coincidence factor}} \cdot \exp\left(-\frac{\|j - \mu_i\|^2}{2\sigma_{\text{eff}}^2}\right),$$

where

$$\mu_i = \frac{\sigma_t^2 r_i + \sigma_r^2 t_i}{\sigma_r^2 + \sigma_t^2}, \quad \sigma_{\text{eff}}^2 = \frac{\sigma_r^2 \sigma_t^2}{\sigma_r^2 + \sigma_t^2}.$$

Proof. Both factors of $\varphi_{i,j}^{\text{STM}}$ are Gaussians in j with precisions $1/\sigma_r^2$ (centered at r_i) and $1/\sigma_t^2$ (centered at t_i). Completing the square in the sum of the two exponents,

$$\frac{\|j - r_i\|^2}{\sigma_r^2} + \frac{\|j - t_i\|^2}{\sigma_t^2} = \frac{\|j - \mu_i\|^2}{\sigma_{\text{eff}}^2} + \frac{\|r_i - t_i\|^2}{\sigma_r^2 + \sigma_t^2},$$

gives the factorization above. $\square$

σ_{eff} depends only on (σ_r, σ_t) , never on r_i or t_i : the fused kernel’s spatial *width* is fixed regardless of how far the BMU sits from its anchor. $\|r_i - t_i\|$ affects only (i) the coincidence factor, a scalar ≤ 1 that discounts the update’s overall magnitude, and (ii) the center μ_i , which shifts away from t_i towards r_i . Lemma 1 therefore holds unconditionally: it does not require $r_i = t_i$, only that both are well-defined grid points, and σ_{eff} is the quantity that governs the true spatial radius of interference once a class is learned, whether or not the BMU has snapped to its anchor.

Corollary 1 (Exact snap). *If $r_i = t_i = a$, the coincidence factor is 1 and $\mu_i = a$: the fused kernel is a single Gaussian centered exactly at the anchor.*

Corollary 2 (Scatter radius). *Let $\kappa := \sigma_t^2/(\sigma_r^2 + \sigma_t^2) \in (0, 1)$. If $\|r_i - t_i\| \leq \rho$ for some scatter radius $\rho \geq 0$ (the BMU lies within ρ grid units of the anchor, without necessarily coinciding with it), then the coincidence factor is bounded below by $\exp(-\rho^2/2(\sigma_r^2 + \sigma_t^2))$, and $\|\mu_i - t_i\| \leq \kappa\rho \leq \rho$.*

Corollary 2 follows directly from the definitions of the coincidence factor and μ_i in Lemma 1. It is the condition actually needed below: ρ is a property of the trained BMU population (the typical grid distance between a class’s own samples’ BMUs and its anchor after training) and is directly measurable, rather than an idealization that either holds exactly or does not apply at all.

S.3 Single-phase learning-leakage bound

The online update of a prototype w_j can be modeled as a leaky integrator, in which each of the T steps of a phase pulls w_j toward the current sample by $\eta \cdot \varphi_{i,j}^{\text{STM}}$. In a phase that trains exclusively on class c (anchor a_c), the expected drift of a prototype at grid distance $d_j = \|j - a_c\|$ obeys the following bound.

Proposition 1 (Single-phase learning-leakage bound).

$$\mathbb{E}[\|\Delta w_j(T)\|] \leq \left(1 - \exp(-T\eta \cdot e^{-d_j^2/2\sigma_{\text{eff}}^2})\right) \cdot \text{diam}(X).$$

The bound decays doubly with grid distance (a Gaussian inside an exponential) and is negligible beyond a few σ_{eff} . As stated, it assumes that at every step of the phase the best-matching unit (BMU) sits exactly on the class anchor, that is, $r_i = t_i = a_c$ (Corollary 1). The next corollary shows that this assumption is not needed.

Corollary 3 (Scatter-robust learning-leakage bound). *If, at every step, the BMU lies within distance ρ of the anchor, $\|r_i - a_c\| \leq \rho$ (Corollary 2), instead of sitting exactly on it, then for a prototype j with $d_j \geq \kappa\rho$,*

$$\mathbb{E}[\|\Delta w_j(T)\|] \leq \left(1 - \exp(-T\eta \cdot e^{-(d_j - \kappa\rho)^2/2\sigma_{\text{eff}}^2})\right) \cdot \text{diam}(X),$$

with κ as in Corollary 2; Proposition 1 is the $\rho = 0$ special case.

Proof. By Lemma 1 and Corollary 2, every step’s kernel weight satisfies $\varphi_{i,j}^{\text{STM}} \leq \exp(-\|j - \mu_i\|^2/2\sigma_{\text{eff}}^2)$ (dropping the coincidence factor, which is ≤ 1), and by the triangle inequality $\|j - \mu_i\| \geq \|j - a_c\| - \|\mu_i - a_c\| \geq d_j - \kappa\rho$. Substituting this upper bound on $\varphi_{i,j}^{\text{STM}}$ into the leaky-integrator argument behind Proposition 1 gives the stated bound. $\square$

The qualitative conclusion, that learning leakage decays doubly with grid distance to the anchor, survives intact under any fixed scatter radius ρ ; ρ only shifts the effective origin of the decay from a_c to a $\kappa\rho$ -ball around it, and is directly measurable from the trained BMU population rather than assumed. It does not, however, cover the transient before a class’s own BMUs first settle within ρ of their anchor.

Corollary 4 (Annealed phase). *Let the neighborhood width $\sigma_r(s)$ change with the step s of a phase, so that $\sigma_{\text{eff}}(s)$ changes too. The leakage onto prototype j then depends on its total exposure, the sum over steps of the kernel weight that j receives at distance d_j from the anchor,*

$$S_j(T) = \sum_{s=1}^T \exp(-d_j^2/2\sigma_{\text{eff}}(s)^2),$$

and, to a good approximation,

$$\mathbb{E}[\|\Delta w_j(T)\|] \lesssim (1 - \exp(-\eta S_j(T))) \cdot \text{diam}(X).$$

Proposition 1 is the special case of a constant σ_{eff} , where $S_j(T) = T e^{-d_j^2/2\sigma_{\text{eff}}^2}$. What matters, then, is not how long a phase lasts but how much kernel weight a distant prototype collects along the way. Early in a phase σ_r is large, the kernel is wide, and prototypes far from the anchor are still pulled along with it. As σ_r shrinks the kernel narrows, and the same prototypes receive almost no weight. Schedules that shrink σ_r quickly (front-loaded) spend most of the phase in the narrow regime, so $S_j(T)$ stays small and little is forgotten. Schedules that keep σ_r large for longer (back-loaded or plateau) let $S_j(T)$ grow, and leakage grows with it.

S.4 Two-term interference: learning leakage and generative-replay’s corrective term

Proposition 1 and Corollaries 3 and 4 bound only *learning leakage*: the incidental drift a prototype suffers from training that targets a *different* anchor. They implicitly assume every step of a phase targets the same anchor a_c . This assumption is false whenever generative replay is active: within a phase nominally training class c , a fraction of steps are decoder-sampled synthetic images for previously-seen classes, generated at jittered positions near *their own* anchors and given identical loss treatment to real data (Section 3.1). The training stream is therefore a mixture over anchors, not a single stream, and the leaky-integrator model must be extended accordingly.

Let p_c be the fraction of a phase’s steps carrying the current class’s own anchor, and $p_{c'}$ the fraction carrying a previously-seen class c' ’s (jittered) anchor. The replay batch of a phase contains γ times as many samples as the current class’s real data, where γ is the replay ratio, and these samples are divided equally among the n previously seen classes. Per unit of real data, the phase therefore carries 1 part current class and γ/n parts for each old class, out of $1 + \gamma$ parts in total. This gives $p_c = 1/(1 + \gamma)$ and $p_{c'} = \gamma/[n(1 + \gamma)]$ for each old class.

Proposition 2 (Two-term interference). *In a phase of T steps that trains class c , the expected drift of a prototype j at (or near) the anchor of an old class c' is approximately*

$$\mathbb{E}[\Delta w_j(T)] \approx \underbrace{p_c(1 - e^{-T\eta\phi(d_j)})(\mu_c - w_j)}_{\text{learning leakage, away from } c'} + \underbrace{p_{c'}(1 - e^{-T\eta\phi(0)})(\mu_{c'} - w_j)}_{\text{refresh, toward } c'}.$$

Here μ_c and $\mu_{c'}$ are the mean samples of the two classes, d_j is the grid distance from j to a_c , and $\phi(d)$ is the fused kernel weight at distance d from an anchor (Lemma 1), equal to $e^{-d^2/2\sigma_{\text{eff}}^2}$ when the BMU sits exactly on the anchor. Each term is the share of steps carrying the relevant anchor, times how far w_j is pulled toward that class’s samples (the leaky-integrator formula of Section S.2), times the distance left to cover. The first term is the leakage bounded above: it

pulls w_j toward the class being trained, and stays small because $\phi(d_j)$ is small far from a_c . The second term is new. The replayed samples of c' itself sit at j ($d = 0$) and pull w_j back toward c' . Forgetting of c' is the balance between the two.

The interference bound derived above constrains only the first term, and it still holds for that term. Replay actually tightens it: once replay is active, $p_c < 1$, so fewer steps train on current-class data alone. What the bound does not give us is a limit on *net* forgetting. Net drift equals learning leakage *minus* refresh, and the shallow-map draft's original analysis has no term for refresh at all.

Proposition 2 holds exactly only when the BMU sits exactly on its anchor for every sample, since only then do $\phi(d_j)$ and $\phi(0)$ take the idealized values used above (Corollary 1). For real samples this condition was relaxed earlier. For replay samples it cannot be expected either: they are generated at a jittered offset from $a_{c'}$, so their BMUs fall within a radius set by the jitter, not on $a_{c'}$ itself, and $\phi(0) = 1$ no longer applies.

Corollary 5 (Scatter-robust two-term interference). *Let ρ be class c 's own real-sample scatter radius (Corollary 3) and ρ_g the replay stream's scatter radius around $a_{c'}$ (Corollary 2, applied to a replay sample's BMU and its own anchor $a_{c'}$ rather than to a real sample). Then*

$$\phi(d_j) \leq e^{-(d_j - \kappa\rho)^2 / 2\sigma_{\text{eff}}^2}, \quad \phi(0) \geq e^{-\rho_g^2 / 2(\sigma_r^2 + \sigma_t^2)} \cdot e^{-\kappa^2 \rho_g^2 / 2\sigma_{\text{eff}}^2},$$

each following from Lemma 1 and Corollary 2 exactly as in the proof of Corollary 3, the second by evaluating the fused kernel's coincidence factor and worst-case spatial term at the prototype $j = a_{c'}$ itself.

The corollary puts numbers on the fact that BMUs land near their anchors, not exactly on them. For the class being trained, the BMUs stay within ρ of a_c , so a prototype at distance d_j is pulled as if it were at distance $d_j - \kappa\rho$, and the leakage term can be larger than in the idealized formula of Proposition 1. For the replay stream, the BMUs stay within ρ_g of $a_{c'}$, so the kernel weight at the prototype itself is $\phi(0) \geq e^{-\rho_g^2 / 2(\sigma_r^2 + \sigma_t^2)} e^{-\kappa^2 \rho_g^2 / 2\sigma_{\text{eff}}^2}$, a value below 1 that decreases as ρ_g grows, and the refresh term is weaker than in the idealized formula. Taking scatter into account therefore raises the leakage term and lowers the refresh term, so net forgetting is larger than the idealized formula gives.

This weakening of the refresh acts from the start. Even in a class's earliest refresh phases, when $p_{c'}$ is still large, the replayed images' BMUs are scattered around $a_{c'}$, and the refresh delivers only the fraction $\phi(0)$ of its idealized value. A second weakening comes from dilution: $p_{c'}$ shrinks as the number n of old classes grows (Corollary 6), most strongly for classes seen early. Both effects reduce the correction that replay provides, in line with the empirical finding of Section 3.2.3 that replay closes about a tenth of the BWT gap left by the spatial mechanism. Separating their contributions requires following each old class phase by phase, using the result matrix R (Section S.9), whose entries give the accuracy on each earlier class right after each phase, instead of the aggregate table of Section 3.2.3.

Corollary 6 (Recency asymmetry). *Because $p_{c'}$ shrinks as $1/n$ while n (the number of previously-seen classes) grows with every subsequent phase, an early-learned class accumulates learning-leakage exposure from more later phases while its own refresh allocation is diluted across more competing classes. The bound therefore predicts that earlier-learned classes forget disproportionately more than a distance-only account would suggest, and that the very last class trained in a run receives no refresh at all (no later phase exists to replay it into). This prediction can be checked against the result matrix R (Section S.9), which records the accuracy on each earlier class after every phase: earlier-learned classes should show the larger drops.*

S.5 Batch and online updates

Fixing r turns minimizing E (Section S.1) into a weighted least-squares problem with a closed-form solution, giving a block-coordinate-descent algorithm that alternates:

E-step for fixed W , recompute $r_i(W)$ for every input i ;

M-step for fixed r , update every prototype to the φ^{STM} -weighted mean

$$w_j \leftarrow \frac{\sum_i \varphi_{i,j}^{\text{STM}} x_i}{\sum_i \varphi_{i,j}^{\text{STM}}}.$$

Theorem 1 (M-step descent). *For fixed σ_r and fixed assignment r , the M-step update satisfies $E^{(t+1)} \leq E^{(t)}$.*

Proof. For fixed r , E is a strictly convex quadratic in W (Section S.1), and the M-step update sets each w_j to the minimiser of its own term of that quadratic, the φ^{STM} -weighted mean of the inputs it is responsible for. $\square$

This guarantee is local to a single, fixed- σ_r epoch. It says nothing about what happens across epoch boundaries as σ_r anneals (Section 3.1): the assignment r , the energy landscape, and the previous minimiser can all change simultaneously when σ_r shrinks, so the monotone-descent property of Theorem 1 does not extend to the annealing schedule as a whole. Empirical convergence under annealing (Section 4) is an observed property of the training procedure, not a corollary of Theorem 1.

An online variant applies the same weighting to a single-sample gradient step rather than the closed-form batch mean:

$$w_j \leftarrow w_j + \eta \varphi_{i,j}^{\text{STM}} (x_i - w_j),$$

which is the update rule the shallow baseline (Figure 3) uses directly, with W stored as an explicit (K^2, D) matrix rather than computed by a network.

S.6 Batch loss for the deep generative decoder

Section S.5 gave the batch M-step for the shallow, table-stored case; this section gives its counterpart for the deep generative decoder (Section 2.2), the architecture actually used in every continual-learning experiment in this paper (the champion configuration of Table 1). Concretely, it derives the loss minimized on each minibatch once a shared decoder network replaces the prototype table W . The loss now has two terms: L_{stm} and a new reconstruction term, L_{recon} .

For a minibatch $\{x_i\}_{i=1}^B$, the E-step is unchanged: each r_i is still the prototype-distance BMU against the current $W = \text{dec}(I_{K^2}; \theta)$, and $\varphi_{i,j}^{\text{STM}}$ is evaluated at that fixed r_i (and, for the training-time value used below, the sample’s own anchor t_i). Because $\varphi_{i,j}^{\text{STM}}$ is a function only of grid positions and the (index-valued, non-differentiable) BMU, it carries no gradient into θ : it plays exactly the role of the M-step’s fixed weighting, only now as a constant multiplier inside a differentiable batch loss rather than inside a closed-form average. The M-step itself is replaced by one gradient step on

$$L_{\text{stm}}(\theta) = \frac{1}{2B} \sum_{i=1}^B \sum_j \varphi_{i,j}^{\text{STM}} \|x_i - \text{dec}(e_j; \theta)\|^2,$$

the same weighted-distortion energy of Section 2.1, batched and averaged rather than summed, with $\varphi_{i,j}^{\text{STM}}$ detached from the computation graph. A second term, absent from the shallow case because the shallow baseline has no shared function to query at an interpolated code,

$$L_{\text{recon}}(\theta) = \frac{1}{BD} \sum_{i=1}^B \|x_i - \text{dec}(\bar{\phi}_i; \theta)\|^2, \quad \bar{\phi}_i = \frac{\varphi_{i,\cdot}^{\text{STM}}}{\sum_j \varphi_{i,j}^{\text{STM}}},$$

scores the decoder’s direct reconstruction of x_i from its own row-normalized φ^{STM} vector, a single fused decoder call rather than a K^2 -term sum. It is a mean over pixels (the input has D

pixels), whereas L_{stm} sums over them, so $\lambda_{\text{recon}} = 0.1$ corresponds to a weight of $0.1/D$ on the per-sample squared norm. The champion configuration’s per-step batch loss is the weighted sum of the two,

$$L(\theta) = \lambda_{\text{stm}} L_{\text{stm}}(\theta) + \lambda_{\text{recon}} L_{\text{recon}}(\theta),$$

with $\lambda_{\text{stm}} = 1.0$ and $\lambda_{\text{recon}} = 0.1$ (Table 1), minimized by Adam rather than assigned in closed form.

The closed-form M-step of Section S.5 works because W is a table of K^2 independent vectors: once r is fixed, $E(W)$ splits into K^2 separate least-squares problems, each solved by its own φ^{STM} -weighted mean. With a deep decoder every prototype is $w_j = \text{dec}(e_j; \theta)$, a function of one shared parameter vector θ , so that split is gone and θ as a whole has no closed-form minimizer. Training therefore takes ordinary gradient steps on $L(\theta)$, which Theorem 1 does not cover. Two exact statements still hold at fixed r , and together they recover much of the shallow structure:

1. the deep M-step is a weighted regression onto the shallow targets $\bar{x}_j$ (Lemma 2), and
2. in the champion decoder, each unit’s latent vector is trained by its own term only, given the shared stack D .

1. Reduction to the shallow targets. Fix r , and write $\Phi_j = \sum_i \varphi_{i,j}^{\text{STM}}$ and $\bar{x}_j = \sum_i \varphi_{i,j}^{\text{STM}} x_i / \Phi_j$, the shallow M-step target of Section S.5.

Lemma 2 (Reduction to $\bar{x}$). *For any decoder and any fixed r ,*

$$L_{\text{stm}}(\theta) = c + \frac{1}{2B} \sum_j \Phi_j \|\bar{x}_j - \text{dec}(e_j; \theta)\|^2, \quad c = \frac{1}{2B} \sum_{i,j} \varphi_{i,j}^{\text{STM}} \|x_i - \bar{x}_j\|^2,$$

where c does not depend on θ .

Proof. Expand $\|x_i - w_j\|^2 = \|x_i - \bar{x}_j\|^2 + 2\langle x_i - \bar{x}_j, \bar{x}_j - w_j \rangle + \|\bar{x}_j - w_j\|^2$. The cross term vanishes after summing over i with weights $\varphi_{i,j}^{\text{STM}}$, because $\sum_i \varphi_{i,j}^{\text{STM}} (x_i - \bar{x}_j) = 0$ by definition of $\bar{x}_j$. $\square$

So the deep M-step is a Φ -weighted regression of the K^2 decoder outputs onto the shallow targets, and it no longer depends on the batch size B . The constant c is the loss of the shallow optimum $W = \bar{x}$, which lower-bounds what any decoder can reach at fixed r ; the distance from c measures what the decoder gives up.

2. Per-unit latents in the champion decoder. The champion projects the grid vector with one linear layer and no bottleneck (Section S.8), so $w_j = D(a_j)$ with $a_j = A_{:,j} + b$, where D is the shared transposed-convolution stack. By Lemma 2, the gradient reaching the free vector $A_{:,j}$ comes only from unit j ’s own term $\frac{\Phi_j}{2B} \|\bar{x}_j - D(a_j)\|^2$. Given D , the K^2 latents are therefore independent problems. This is the gating of Section S.7: the latents are gated, and D is the ungated shared part.

S.7 Which parameters Topographic Gating actually reaches

Only the decoder’s first layer is gated by grid position; every layer downstream of it, and every convolutional layer in the upsampling stack, receives an undamped training signal on every step, regardless of which unit is currently being trained. The decoder’s first layer is $y_1 = W_1 z_i + b_1$. Because z_i is indexed by grid unit, the gradient reaching one specific column of W_1 (the column belonging to unit j) is

$$\frac{\partial \mathcal{L}}{\partial W_1[:, j]} \propto z_i[j].$$

This is nothing more than the chain rule for a linear layer, but it has a direct consequence: if unit j is far from wherever sample i is training, $z_i[j]$ is numerically zero, and so is the gradient reaching that column. Column j of W_1 is, in this precise sense, *gated* by the anchor kernel: it has a dedicated slot that only unit j ’s own φ^{STM} value can activate, exactly as the shallow baseline’s prototype row j does (Section 1.2.3).

No layer downstream of W_1 has this property, and the reason is structural rather than incidental. The first layer’s output, $h = \text{SiLU}(y_1)$, is a generic hidden vector: every one of its entries mixes contributions from whichever units were active in z_i , and none of its coordinates is dedicated to any single unit. The gradient reaching the next layer’s weights, W_2 , is proportional to h itself, not to any per-unit quantity: there is no column of W_2 that a distant, currently-untrained unit could claim as its own protected slot, because that slot was never created. Gating requires a parameter to be multiplied directly by that unit’s own φ^{STM} value; once one layer has compressed a K^2 -dimensional, unit-indexed input into a lower-dimensional, unit-agnostic code, every later layer receives a training signal at full strength on every step, regardless of which unit is currently being trained.

The convolutional layers in the decoder’s upsampling stack are ungated for a stronger reason still: a convolution kernel is, by construction, a single small filter reused at every spatial position of a feature map, and was never indexed by grid unit in the first place. This holds regardless of how many linear layers precede it, or how wide they are: the moment a K^2 -dimensional grid vector has been projected into a feature map, no operation downstream can recover which unit any part of that map came from, and every parameter from that point on is trained on whatever the network currently sees, without Topographic Gating’s protection.

S.8 Architecture ablation: decoder width and channel count

Section 2.2 left the channel count of the decoder’s convolutional stage as an open architectural choice, and reported that any hidden bottleneck between the grid vector and the convolutional stage is decisively worse than a direct single-layer projection. Both are confirmed here, with every other hyperparameter fixed at the value given in Section 3.1.3.

Decoder width. Routing the K^2 -dimensional grid vector through a hidden bottleneck of width h before the convolutional stage, instead of a direct single dense layer ($h = 0$, no bottleneck), is decisive and uniformly bad:

Bottleneck width h	0	32	64	128	256
AA (seed 42)	0.9079	0.8032	0.7907	0.8149	0.8074

The ~ 10 -point drop is an order of magnitude larger than any margin observed across every other hyperparameter tested for this paper, has no dependence on bottleneck width, and needed no multi-seed confirmation. Every unit’s prototype is $w_j = \text{dec}(e_j)$; a hidden bottleneck of width h caps the rank of the map from 625 one-hot inputs to 625 prototypes at h , *before* the nonlinearity, which is enough to collapse prototype diversity regardless of how wide h is.

Decoder channel count. Unlike width, the convolutional stage’s channel count c shows a real, non-monotonic optimum:

Channel count c	16	20	24	28	32	36	40	44	48
AA	0.9043	0.9055	0.9087	0.9053	0.9046	0.9052	0.9021	0.9027	0.8980
std	0.0127	0.0075	0.0051	0.0076	0.0066	0.0057	0.0141	0.0077	0.0150
outliers (AA < 0.88)	2/25	0/25	0/25	0/25	0/25	0/25	1/25	0/25	1/25

All nine values at $n = 25$ seeds each. $c = 24$ is an isolated single-point peak, not the edge of a broad plateau: its immediate neighbors (20, 28) and even 36 sit on a flat plateau roughly 0.003 to 0.004 lower, while instability (elevated variance, outliers) reappears asymmetrically at 16 on the low side and 40/48 on the high side. Against the previously used $c = 32$ ($AA = 0.9046 \pm 0.0066$), $c = 24$'s margin is $\Delta = +0.0041$ with $t \approx 2.4$: the most credible alternative found, but still short of a range-overlap-free confirmation. It is nonetheless adopted, and is the value used throughout Sections 3.2 and 4.

S.9 Metrics

Let $R \in \mathbb{R}^{T \times T}$ be the standard continual-learning result matrix (Lopez-Paz and Ranzato, 2017): $R[i][j]$ is accuracy on task j 's test set measured immediately after training on task i ($T = 5$ here). Two summary metrics from that same convention are reported in the main text:

$$AA = \frac{1}{T} \sum_{i=0}^{T-1} R[T-1][i], \quad BWT = \frac{1}{T-1} \sum_{i=0}^{T-2} (R[T-1][i] - R[i][i]).$$

A third metric, Class-IL, reports a single pooled accuracy over the full ten-class test set evaluated after all training completes, rather than the unweighted per-task average AA uses; the two are close but not identical, since Class-IL implicitly reweights by each class's share of the pooled test set rather than each task's share of the task average.

A fourth metric from the same suite, forward transfer, is defined as

$$FWT = \frac{1}{T-1} \sum_{i=1}^{T-1} (R[i-1][i] - \text{chance}), \quad \text{chance} = 0.5,$$

using the same result matrix R , with $\text{chance} = 0.5$ the correct random baseline for each phase's own binary (two-digit) test subset. This paper omits FWT from its main-text results because, under majority-vote best-matching-unit labeling (Section 3.1.1), it is a structural constant rather than a measurement: at the moment $R[i-1][i]$ is measured, task i 's classes have not yet been trained, so no grid unit can yet carry one of their labels, giving $R[i-1][i] = 0$ for every phase transition and every seed, and hence $FWT = 0 - 0.5 = -0.5000$ with zero variance across all seeds tested. This is a property of the labeling rule's bookkeeping, not evidence about the model's true zero-shot transfer, and is recorded here only so that FWT's absence from the main text is not mistaken for an oversight.

References

References

- Amari, S. (1977). Dynamics of pattern formation in lateral-inhibition type neural fields. *Biological Cybernetics*, 27(2):77–87.
- Arcaro, M. J. and Livingstone, M. S. (2024). A whole-brain topographic ontology. *Annual Review of Neuroscience*, 47:21–40.
- Argunşah, A. O., Stachniak, T. J., Yang, J.-W., Cai, L., van der Bourg, A., Kastli, R., and Karayannis, T. (2026). Local inhibitory dynamics underpin temporal integration and functional segregation between barrels and septa in the mouse barrel cortex. *eLife*, 14:RP107099.
- Barack, D. L. and Krakauer, J. W. (2021). Two views on the cognitive brain. *Nature Reviews Neuroscience*, 22(6):359–371.

- Borzello, M., Ramirez, S., Treves, A., Lee, I., Scharfman, H., Stark, C., Knierim, J. J., and Rangel, L. M. (2023). Assessments of dentate gyrus function: Discoveries and debates. *Nature Reviews Neuroscience*, 24(8):502–517.
- Cayco-Gajic, N. A. and Silver, R. A. (2019). Re-evaluating circuit mechanisms underlying pattern separation. *Neuron*, 101(4):584–602.
- Chaudhry, A., Ranzato, M., Rohrbach, M., and Elhoseiny, M. (2019). Efficient lifelong learning with a-gem. In *International Conference on Learning Representations*.
- Chaudhuri, R., Gercek, B., Pandey, B., Peyrache, A., and Fiete, I. (2019). The intrinsic attractor manifold and population dynamics of a canonical cognitive circuit across waking and sleep. *Nature Neuroscience*, 22(9):1512–1520.
- Cline, H. T., Lau, M., and Hiramoto, M. (2023). Activity-dependent organization of topographic neural circuits. *Neuroscience*, 508:3–18.
- Constantinescu, A. O., O’Reilly, J. X., and Behrens, T. E. J. (2016). Organizing conceptual knowledge in humans with a gridlike code. *Science*, 352(6292):1464–1468.
- Deja, K., Wawrzyński, P., Marczak, D., Masarczyk, W., and Trzciński, T. (2020). Binplay: A binary latent autoencoder for generative replay continual learning. *arXiv:2011.14960*.
- Erlhagen, W. and Schöner, G. (2002). Dynamic field theory of movement preparation. *Psychological Review*, 109(3):545–572.
- Gardner, R. J., Hermansen, E., Pachitariu, M., Burak, Y., Baas, N. A., Dunn, B. A., Moser, M.-B., and Moser, E. I. (2022). Toroidal topology of population activity in grid cells. *Nature*, 602:123–128.
- Georgopoulos, A. P., Schwartz, A. B., and Kettner, R. E. (1986). Neuronal population coding of movement direction. *Science*, 233(4771):1416–1419.
- Gross, C. G. (2002). Genealogy of the “grandmother cell”. *The Neuroscientist*, 8(5):512–518.
- Ha, D., Dai, A., and Le, Q. V. (2016). HyperNetworks. *arXiv:1609.09106*.
- Harvey, B. M., Klein, B. P., Petridou, N., and Dumoulin, S. O. (2013). Topographic representation of numerosity in the human parietal cortex. *Science*, 341(6150):1123–1126.
- Hu, Z. and Rostami, M. (2023). Class-incremental learning using generative experience replay based on time-aware regularization. *arXiv:2310.03898*.
- Huntenburg, J. M., Bazin, P.-L., and Margulies, D. S. (2018). Large-scale gradients in human cortical organization. *Trends in Cognitive Sciences*, 22(1):21–31.
- Huth, A. G., de Heer, W. A., Griffiths, T. L., Theunissen, F. E., and Gallant, J. L. (2016). Natural speech reveals the semantic maps that tile human cerebral cortex. *Nature*, 532(7600):453–458.
- Kaas, J. H. (1997). Topographic maps are fundamental to sensory processing. *Brain Research Bulletin*, 44(2):107–112.
- Kingma, D. P. and Welling, M. (2013). Auto-encoding variational bayes. *arXiv:1312.6114*.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.

- Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69.
- Kohonen, T. (2001). *Self-Organizing Maps*, volume 30 of *Springer Series in Information Sciences*. Springer, 3rd edition.
- Lee, H., Wang, C., Deshmukh, S. S., and Knierim, J. J. (2015). Neural population evidence of functional heterogeneity along the CA3 transverse axis: Pattern completion versus pattern separation. *Neuron*, 87(5):1093–1105.
- Li, Z. and Hoiem, D. (2017). Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947.
- Lin, A. C., Bygrave, A. M., de Calignon, A., Lee, T., and Miesenböck, G. (2014). Sparse, decorrelated odor coding in the mushroom body enhances learned odor discrimination. *Nature Neuroscience*, 17:559–568.
- Lopez-Paz, D. and Ranzato, M. (2017). Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30.
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics*, pages 281–297. University of California Press.
- Makhzani, A., Shlens, J., Jaitly, N., Goodfellow, I., and Frey, B. (2015). Adversarial autoencoders. *arXiv:1511.05644*.
- Mallya, A. and Lazebnik, S. (2018). Packnet: Adding multiple tasks to a single network by iterative pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7765–7773.
- Mannella, F. (2020). Supervised topological maps. *arXiv:2008.06395*.
- Mannella, F. and Tummolini, L. (2023). Kickstarting the grounding of concepts with topological alignment. *Philosophical Transactions of the Royal Society B*, 378:20210370.
- Margulies, D. S., Ghosh, S. S., Goulas, A., Falkiewicz, M., Huntenburg, J. M., Langs, G., Bezgin, G., Eickhoff, S. B., Castellanos, F. X., Petrides, M., Jefferies, E., and Smallwood, J. (2016). Situating the default-mode network along a principal gradient of macroscale cortical organization. *Proceedings of the National Academy of Sciences*, 113(44):12574–12579.
- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. (2020). NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision*, pages 405–421.
- Müller, T., Evans, A., Schied, C., and Keller, A. (2022). Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):102:1–102:15.
- Nieh, E. H., Schottdorf, M., Freeman, N. W., Low, R. J., Lewallen, S., Koay, S. A., Pinto, L., Gauthier, J. L., Brody, C. D., and Tank, D. W. (2021). Geometry of abstract learned knowledge in the hippocampus. *Nature*, 595:80–84.
- Park, J. J., Florence, P., Straub, J., Newcombe, R., and Lovegrove, S. (2019). DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 165–174.
- Pinitas, K., Chavlis, S., and Poirazi, P. (2021). Dendritic self-organizing maps for continual learning. *arXiv:2110.13611*.

- Po, R., Dong, Z., Bergman, A. W., and Wetzstein, G. (2023). Instant continual learning of neural radiance fields. In *IEEE/CVF International Conference on Computer Vision Workshops*.
- Pouget, A., Dayan, P., and Zemel, R. (2000). Information processing with population codes. *Nature Reviews Neuroscience*, 1:125–132.
- Quiroga, R. Q., Reddy, L., Kreiman, G., Koch, C., and Fried, I. (2005). Invariant visual representation by single neurons in the human brain. *Nature*, 435(7045):1102–1107.
- Rebuffi, S.-A., Kolesnikov, A., Sperl, G., and Lampert, C. H. (2017). icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- Serra, J., Suris, D., Miron, M., and Karatzoglou, A. (2018). Overcoming catastrophic forgetting with hard attention to the task. In *International Conference on Machine Learning*, pages 4548–4557.
- Shin, H., Lee, J. K., Kim, J., and Kim, J. (2017). Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*.
- Stensola, H., Stensola, T., Solstad, T., Frøland, K., Moser, M.-B., and Moser, E. I. (2012). The entorhinal grid map is discretized. *Nature*, 492:72–78.
- Straub, I., Witter, L., Eshra, A., Hoidis, M., Byczkiewicz, N., Maas, S., Delvendahl, I., Dorgans, K., Savier, E., Bechmann, I., Krueger, M., Isope, P., and Hallermann, S. (2020). Gradients in the mammalian cerebellar cortex enable Fourier-like transformation and improve storing capacity. *eLife*, 9:e51771.
- Thapa, P., Ororbia, A., and Desell, T. (2025). Class incremental continual learning with self-organizing maps and synthetic replay. *arXiv:2508.21240*.
- Urbanik, I. and Gajewski, P. (2025). Satsom: Saturation self-organizing maps for continual learning. *arXiv:2506.10680*.
- van de Ven, G. M., Siegelmann, H. T., and Tolias, A. S. (2020). Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11(1):4069.
- van de Ven, G. M. and Tolias, A. S. (2018). Generative replay with feedback connections as a general strategy for continual learning. *arXiv:1809.10635*.
- van de Ven, G. M. and Tolias, A. S. (2019). Three scenarios for continual learning. *arXiv:1904.07734*.
- Vyas, S., Golub, M. D., Sussillo, D., and Shenoy, K. V. (2020). Computation through neural population dynamics. *Annual Review of Neuroscience*, 43:249–275.
- Wallace, M. T. and Stein, B. E. (2001). Sensory and multisensory responses in the newborn monkey superior colliculus. *Journal of Neuroscience*, 21(22):8886–8894.
- Wang, Y., Wang, J., Wang, C., Duan, W., Bao, Y., and Qi, Y. (2024). SCARF: Scalable continual learning framework for memory-efficient multiple neural radiance fields. *Computer Graphics Forum*, 43(7).
- Wei, X.-X., Prentice, J., and Balasubramanian, V. (2015). A principle of economy predicts the functional architecture of grid cells. *eLife*, 4:e08362.
- Wilson, H. R. and Cowan, J. D. (1972). Excitatory and inhibitory interactions in localized populations of model neurons. *Biophysical Journal*, 12(1):1–24.

- Xie, Y., Takikawa, T., Saito, S., Litany, O., Yan, S., Khan, N., Tombari, F., Tompkin, J., Sitzmann, V., and Sridhar, S. (2022). Neural fields in visual computing and beyond. *Computer Graphics Forum*, 41(2):641–676.
- Yassa, M. A. and Stark, C. E. L. (2011). Pattern separation in the hippocampus. *Trends in Neurosciences*, 34(10):515–525.
- Yeates, D. C. M., Ussling, A., Lee, A. C. H., and Ito, R. (2020). Double dissociation of learned approach–avoidance conflict processing and spatial pattern separation along the dorsoventral axis of the dentate gyrus. *Hippocampus*, 30:596–609.
- Zenke, F., Poole, B., and Ganguli, S. (2017). Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, pages 3987–3995.
- Zubek, K., Mannella, F., and Tummolini, L. (2025). Learning flexible action control through topological alignment. Preprint.